

408 数据结构学习笔记

第一章 绪论

本章导学：本章学什么、考什么

考纲内容与复习重心

本章主要解决两个问题：**数据结构是什么**，以及**怎样评价算法好不好**。前者是后续线性表、栈、队列、树、图等内容的共同语言；后者是考研数据结构中最常出现的分析能力，尤其是时间复杂度和空间复杂度。

本章考纲可以概括为两部分：

1. 数据结构的基本概念

- 数据、数据元素、数据对象、数据类型、抽象数据类型。
- 数据结构的定义。
- 数据结构三要素：逻辑结构、存储结构、数据的运算。

2. 算法的基本概念与算法评价

- 算法定义。
- 算法五个特性：有穷性、确定性、可行性、输入、输出。
- 算法效率评价：时间复杂度和空间复杂度。

从考试角度看，本章最核心的不是背概念，而是要形成一条主线：

数据元素之间有什么关系 → 选择什么逻辑结构
逻辑结构如何落到内存中 → 选择什么存储结构
在这种结构上做什么操作 → 设计对应算法
算法运行得快不快、占空间多不多 → 分析时间复杂度和空间复杂度

知识框架速览

```
第一章 绪论
├─ 数据结构
│   ├── 逻辑结构
│   │   ├── 线性结构：线性表、栈、队列
│   │   └─ 非线性结构：树、图、集合
│   ├── 存储结构
│   │   ├── 顺序存储
│   │   ├── 链式存储
│   │   ├── 索引存储
│   │   └─ 散列存储
│   └─ 数据的运算
└─ 算法
    ├── 算法定义
    ├── 五个特性：有穷性、确定性、可行性、输入、输出
    └─ 效率度量
```

复习时要特别注意：**逻辑结构不等于存储结构**。逻辑结构描述的是数据元素之间的关系；存储结构描述的是这种关系在计算机内存中的表示方式。算法设计首先看逻辑结构，算法实现则离不开具体存储结构。

1.1 数据结构的基本概念

1.1.1 基本概念和术语

这一小节容易被误以为只是名词解释，但它实际上是在建立数据结构课程的“语言系统”。后面说线性表、树、图、栈、队列，本质上都是在讨论数据元素、数据元素之间的关系，以及这些关系在计算机中的表示和操作。

1. 数据

数据是信息的载体，是能被计算机程序识别、存储和处理的符号集合。它不只包括数字，也包括字符、图像编码、音频编码、视频编码等。

可以这样理解：

现实世界的信息 → 被符号化、编码化 → 形成计算机可以处理的数据

例如，学生成绩表中的姓名、学号、性别、课程成绩都可以看作数据。数据是计算机程序加工的原

2. 数据元素

数据元素是数据的基本单位，通常作为一个整体进行考虑和处理。一个数据元素可以由若干个数据项组成。

例如，一条学生记录可以作为一个数据元素：

数据元素	组成的数据项
一条学生记录	学号、姓名、性别、年龄、成绩等

这里要注意层次关系：

数据 > 数据元素 > 数据项

- **数据**：整体集合，例如全体学生信息。
- **数据元素**：集合中的一个基本单位，例如某一个学生的完整记录。
- **数据项**：构成数据元素的不可分割的最小单位，例如学号、姓名、性别。

3. 数据对象

数据对象是具有相同性质的数据元素的集合，是数据的一个子集。

例如，整数数据对象可以表示为集合：

$$N = \{0, \pm 1, \pm 2, \dots\}$$

再比如，“某班全体学生记录”也可以看作一个数据对象，因为这些记录都具有相同性质：都描述学生，都由相似的数据项组成。

4. 数据类型

数据类型是一个值的集合，以及定义在这个值集合上的一组操作的总称。它关心两个问题：

- 1. 这个类型能取哪些值。
- 2. 这个类型允许做哪些操作。

常见数据类型可以分为三类：

类型	含义	例子
原子类型	值不可再分的数据类型	整型、字符型、布尔型
结构类型	值可以再分解为若干成分	数组、结构体、记录
抽象数据类型	从逻辑意义上定义数据和操作，不强调具体实现	栈、队列、线性表

抽象数据类型常写作 ADT。它强调“这个数据对象是什么、能做什么”，而不是“底层到底怎么存”。例如栈的核心是后进先出，只要能支持入栈、出栈、取栈顶等操作，底层既可以用数组实现，也可以用链表实现。

5. 数据结构

数据结构是相互之间存在一种或多种特定关系的数据元素的集合。

这个定义的关键不是“数据元素的集合”，而是“数据元素之间存在关系”。如果只把若干数据元素堆在一起，没有关系，就很难谈得上结构。

数据结构包括三方面内容：

数据结构 = 逻辑结构 + 存储结构 + 数据的运算

组成部分	关注点	简单理解
逻辑结构	数据元素之间的逻辑关系	谁和谁有关系
存储结构	逻辑关系在计算机中的表示	在内存里怎么放、怎么表示关系
数据的运算	对数据元素可以进行哪些操作	查找、插入、删除、遍历等

三者关系非常重要：

- **算法设计**取决于逻辑结构，因为要先知道数据元素之间有什么关系。
- **算法实现**取决于存储结构，因为同一种逻辑结构使用不同存储方式，操作代价可能完全不同。
- **数据运算**既要从逻辑层面定义功能，也要从存储层面落实为具体步骤。

例如，同样是线性表：

逻辑结构	存储结构	插入删除特点
线性表	顺序表	查找方便，但中间插入删除可能需要移动大量元素
线性表	链表	插入删除灵活，但按位置访问不如顺序表方便

这说明：逻辑结构相同，存储结构不同，算法效率也会不同。

1.1.2 数据结构三要素

数据结构三要素是本章最重要的概念之一：**逻辑结构、存储结构、数据的运算**。后续每一种具体结构都可以用这三问来学习：

它的逻辑关系是什么？
它可以怎样存储？
它支持哪些操作，这些操作效率如何？

1. 数据的逻辑结构

逻辑结构是从逻辑角度描述数据元素之间的关系，与它们在计算机中的实际存放位置无关。

逻辑结构主要分为两大类：

数据的逻辑结构

- ├ 线性结构
 - | └ 数据元素之间通常是一对一关系
- └ 非线性结构
 - ├ 集合：元素之间除同属一个集合外无其他关系
 - ├ 树形结构：元素之间存在一对多关系
 - └ 图状结构：元素之间存在多对多关系

四类基本逻辑关系可以这样比较：

逻辑结构	元素关系	典型例子	学习提示
集合	元素之间除“同属一个集合”外无其他关系	整数集合、学生集合	关系最弱，只强调归属
线性结构	元素之间存在一对一关系	线性表、栈、队列、串	有前驱后继关系
树形结构	元素之间存在一对多关系	树、二叉树	分层、父子关系明显
图状结构	元素之间存在多对多关系	有向图、无向图、网	关系最复杂

用关系强弱理解：

集合：只有归属关系
线性结构：一个前驱、一个后继为主
树形结构：一个父结点可以对应多个孩子结点
图状结构：任意两个结点之间都可能有关联

考试中常见判断点：

- 栈、队列、串、数组通常属于线性结构。

- 树、二叉树属于非线性结构。
- 图、网属于非线性结构。
- 集合也属于非线性结构，但它的元素关系最弱。

2. 数据的存储结构

存储结构又称物理结构，是数据结构在计算机中的表示形式。它不仅要存储数据元素本身，还要表示数据元素之间的关系。

常见存储结构有四类：顺序存储、链式存储、索引存储、散列存储。

存储结构	基本思想	优点	缺点
顺序存储	把逻辑上相邻的元素存放在物理位置也相邻的存储单元中	可随机访问，存储空间额外开销小	需要连续空间，插入删除可能移动大量元素，容易产生外部碎片
链式存储	用指针表示元素之间的逻辑关系，物理位置可以不相邻	不要求连续空间，插入删除灵活，不易产生碎片	指针占用额外空间，通常不能随机访问
索引存储	在存储元素信息的同时建立索引表	检索速度快	索引表占用额外空间，增删数据时维护成本高
散列存储	根据关键字通过散列函数直接计算存储地址	查找、插入、删除通常很快	散列函数设计不当会产生冲突，处理冲突有额外成本

其中最需要区分的是顺序存储和链式存储：

顺序存储：靠“物理相邻”体现逻辑相邻

链式存储：靠“指针”体现逻辑关系

因此，顺序表支持按下标快速访问，而链表更适合频繁插入和删除。这种差异会在后续线性表章节中反复出现。

3. 数据的运算

数据的运算包括两个层面：

1. **运算的定义**：基于逻辑结构，说明这个运算要完成什么功能。
2. **运算的实现**：基于存储结构，说明具体如何一步步完成。

例如，线性表的“插入”操作：

层面	关注内容
逻辑定义	在第 i 个位置插入一个新元素，使原有元素次序仍然成立
顺序表实现	可能需要从后往前移动元素，为新元素腾出位置
链表实现	只需要修改相关结点的指针关系

这说明：同一个逻辑运算，在不同存储结构上的实现方式和效率可能完全不同。

1.2 算法和算法评价

1.2.1 算法的基本概念

算法是对特定问题求解步骤的描述，是一个有穷的指令序列，其中每条指令表示一个或多个操作。
可以把算法理解为：

输入若干数据 → 按确定步骤处理 → 在有限时间内得到输出

一个有效算法必须具备五个重要特性。

特性	含义	容易误解的点
有穷性	算法必须在执行有限步后结束，并且每一步都能在有限时间内完成	有穷性强调“必定停机”，不是理论上一直运行
确定性	每条指令含义明确，对于相同输入只能产生相同输出	不能出现含糊不清、随机无法控制的步骤
可行性	算法中的基本操作可以通过已经实现的基本运算在有限次内完成	操作不能只停留在想象中，必须可执行
输入	算法可以有零个或多个输入	没有输入也可以是算法，例如输出固定表格
输出	算法至少有一个输出	没有输出就无法体现求解结果

记忆时可以抓住关键词：

有穷：会停
确定：不歧义
可行：能执行
输入：可有可无
输出：至少一个

通常，设计一个“好”的算法还要尽量满足以下目标：

- 1. **正确性**：能够正确解决问题。这是算法最基本的要求。
- 2. **可读性**：结构清晰、便于理解，便于调试、维护和交流。
- 3. **健壮性**：对非法输入或异常情况能进行适当处理，而不是产生不可预测的结果。
- 4. **高效率与低存储量需求**：运行时间尽量少，额外占用空间尽量少。

这四个目标中，正确性是前提；效率是考试中最常考的评价点。实际分析算法时，通常不会只看程序在某台机器上跑了几秒，而是抽象分析它随问题规模增长时所需时间和空间的^{增长趋势}，这就引出下一小节的时间复杂度和空间复杂度。

1.2.2 算法效率的度量

算法正确只是基础，真正比较算法优劣时，还要看它在问题规模不断增大时，运行时间和占用空间如何增长。由于实际运行时间会受到硬件、编译器、操作系统、测试数据等因素影响，不能直接把

“某台机器上运行了几秒”作为通用标准。因此，数据结构课程通常用**渐近复杂度**来抽象描述算法效率。

考点追踪：（算法题）分析时空复杂度（2010—2013，2015，2016，2018—2021，2025）

1. 时间复杂度

考点追踪：分析算法的时间复杂度（2011—2014，2017，2019，2022，2025）

时间复杂度描述算法执行时间随问题规模 n 增大而变化的趋势。实际分析时，一般不逐条计算所有语句的真实耗时，而是统计基本运算的执行次数。

设算法中所有语句的频度之和为 $T(n)$ ，它表示问题规模为 n 时基本操作执行次数的数量级。只要抓住 $T(n)$ 中增长最快的项，低阶项和常数因子通常都可以忽略。

大 O 记号的含义是：若存在正常数 C 和 n_0 ，使得对所有 $n \geq n_0$ ，都有

$$0 \leq T(n) \leq C f(n)$$

则称

$$T(n) = O(f(n))$$

它表示当 n 足够大时， $T(n)$ 的增长速度不超过 $f(n)$ 的常数倍。通俗理解：大 O 关心的是增长级别，不关心具体常数。

例如：

执行次数函数	时间复杂度	忽略内容
$T(n) = 3n^2 + 100n + 5$	$O(n^2)$	常数因子 3、低阶项 $100n$ 、常数项 5
$T(n) = 2^n + n^{100}$	$O(2^n)$	当 n 足够大时，指数项增长更快

分析时间复杂度时，关键是找出**基本运算**。基本运算通常是算法中执行次数最多、最能体现增长趋势的操作，常出现在最深层循环体中。

例如，在数组 $A[0 \dots n - 1]$ 中从后向前查找值 k ：

```
(1) i = n - 1;
(2) while (i >= 0 && A[i] != k)
(3)     i--;           // 基本运算
(4) return i;
```

这个算法的运行时间与输入数据的初始状态有关：

情况	触发条件	语句 $i--$ 执行次数	时间复杂度理解
最好情况	$A[n - 1] = k$	0	一次就命中，代价最小

情况	触发条件	语句 i — 执行次数	时间复杂度理解
最坏情况	k 不存在, 或在 $A[0]$	n 量级	要从后向前查完整个数组
平均情况	各输入等概率出现	期望执行次数	取所有输入情况下的期望代价

通常使用**最坏时间复杂度**作为算法效率的评价标准, 因为它给出了运行时间的确定性上界。考研中如果没有特别说明, 一般默认分析最坏情况。

对由多段代码组成的程序, 可以用两个规则快速判断整体复杂度:

规则	适用场景	结论	直观理解
加法规则	两段代码顺序执行	$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$	顺序执行看增长更快的部分
乘法规则	一段代码嵌套在另一段内部	$O(f(n)) \cdot O(g(n)) = O(f(n)g(n))$	内层每执行一轮, 外层都要带着它执行

例如, 设 $a\{\}$ 、 $b\{\}$ 、 $c\{\}$ 三个语句块的时间复杂度分别为 $O(1)$ 、 $O(n)$ 、 $O(n^2)$ 。

顺序结构:

```
a{
  b{
  c{
}
```

整体复杂度取最高阶项, 为 $O(n^2)$ 。

嵌套结构:

```
a{
  b{
    c{
  }
}
```

整体复杂度为 $O(1) \cdot O(n) \cdot O(n^2) = O(n^3)$ 。

常见时间复杂度按增长速度从低到高排列为:

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

这条链非常重要。做选择题时, 很多题不是让你精确数出每一步, 而是判断哪个复杂度更高、哪个算法更优。要特别注意: $O(n \log n)$ 比 $O(n^2)$ 增长慢得多; 指数级 $O(2^n)$ 和阶乘级 $O(n!)$ 通常只适合很小规模的问题。

2. 空间复杂度

空间复杂度 $S(n)$ 表示算法在运行过程中所需的额外存储空间随问题规模 n 增大而变化的趋势, 记为

$$S(n) = O(g(n))$$

程序执行时需要的空间大致包括：

1. 指令、常数和变量。
2. 输入数据所占空间。
3. 辅助空间，例如递归调用栈、临时数组、临时指针等。

在分析空间复杂度时，重点通常不是输入数据本身，因为输入数据大小取决于问题规模，不能算作算法额外消耗；也不是与 n 无关的常量变量，因为它们只带来固定开销。真正要关注的是**算法额外使用的辅助空间**。

例如：

情况	空间复杂度	说明
只使用若干个普通变量	$O(1)$	额外空间不随 n 增长，称为原地工作
新建长度为 n 的辅助数组	$O(n)$	额外空间与输入规模同阶增长
递归深度为 n	通常为 $O(n)$	每层递归调用都要占用栈帧空间

空间复杂度常和时间复杂度形成取舍关系。例如，有些算法用额外数组保存中间结果，可以减少重复计算、提升速度，但会增加空间消耗。考研中分析空间复杂度时，尤其要看是否使用了辅助数组、递归栈或额外链表。

3. 复杂度分析的典型方法

本章的核心在于时间复杂度分析。很多题目不能只靠“看一眼”得出答案，而要能把循环次数和问题规模 n 建立关系。

对于循环主体中的变量参与循环条件的程序，应先设基本运算执行次数为 t ，再建立 t 与 n 的关系。

例 1：

```
int i = 1;
while (i <= n)
    i = i * 2;
```

基本运算 $i = i * 2$ 执行 t 次后，有 $2^t \leq n$ ，所以

$$t \leq \log_2 n$$

故时间复杂度为 $O(\log_2 n)$ 。这类“每次乘以常数”的循环，通常是对数级复杂度。

例 2：

```
int y = 5;
while ((y + 1) * (y + 1) < n)
    y = y + 1;
```

设基本运算 $y = y + 1$ 执行 t 次，则此时 $y = t + 5$ 。循环条件近似满足

$$(t + 5 + 1)(t + 5 + 1) < n$$

即

$$(t + 6)^2 < n$$

所以

$$t < \sqrt{n} - 6$$

故时间复杂度为 $O(\sqrt{n})$ 。这类题的关键是不要只看有一个 while 就判断为 $O(n)$ ，而要看循环变量怎样逼近终止条件。

对于循环主体中的变量与循环条件无关的程序，可以直接累加循环执行次数；如果是递归程序，则常用递推式描述。

例如某递归程序每次把规模从 n 减少到 $n - 1$ ，且每层只做常量工作，则有

$$\begin{aligned} T(n) &= 1 + T(n - 1) \\ &= 1 + 1 + T(n - 2) \\ &= \dots \\ &= n - 1 + T(1) \end{aligned}$$

因此 $T(n) = O(n)$ 。

复杂度题的常用判断流程可以总结为：

先找基本操作
再看基本操作由谁控制
如果由循环控制：建立循环次数与 n 的关系
如果由递归控制：写出递推式并求数量级
最后只保留增长最快的项

4. 思维拓展：斐波那契数列的复杂度对比

斐波那契数列定义为：

$$F(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F(n - 1) + F(n - 2), & n > 1 \end{cases}$$

它非常适合用来比较“递归思路”和“非递归思路”的效率差异。

朴素递归写法如下：

```
int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
```

```
    return fib(n - 1) + fib(n - 2);  
}
```

它的缺点是会大量重复计算。例如计算 $F(n)$ 时要计算 $F(n - 1)$ 和 $F(n - 2)$ ，而计算 $F(n - 1)$ 时又会再次计算 $F(n - 2)$ 。递归调用树不断分叉，时间复杂度通常按指数级理解，可记为 $O(2^n)$ 的量级。它的递归深度为 n ，所以空间复杂度为 $O(n)$ 。

非递归迭代写法如下：

```
int fib(int n) {  
    if (n == 0) return 0;  
    int a = 0, b = 1;  
    for (int i = 2; i <= n; i++) {  
        int c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

它从小到大依次计算，每个值只计算一次，因此时间复杂度为 $O(n)$ 。如果只保存最近两个状态，空间复杂度为 $O(1)$ ；如果用数组保存所有 $F(0) \dots F(n)$ ，空间复杂度则为 $O(n)$ 。

这个例子说明：算法思路不同，效率可能差别极大。同样是求 $F(n)$ ，朴素递归思路直观但重复计算严重，迭代或动态规划思路更适合实际计算。

当前进度：已处理《数据结构-第1-5章-绪论至树二叉树.pdf》第 9-25 页，第二章“线性表”的正文、归纳总结与思维拓展均已整理完成；第 26 页已进入第三章，不属于本 act 范围。

下次任务：无。第二章内容已完成，后续不再追加本章内容；按 AnyWorkflow 结束流程完成最终产物上传与当前 act 收尾。

第二章 线性表

本章围绕“线性表”展开。线性表是数据结构中最基础、也最常考的结构之一，后面的栈、队列、串、数组、树和图中的许多操作思想，都会反复用到线性结构的思维。

从考试角度看，本章重点不是背概念，而是掌握两类存储实现：

实现方式	典型结构	核心优势	核心短板
顺序存储	顺序表	随机访问快，能通过下标直接定位元素	插入、删除通常要移动大量元素；要求连续空间
链式存储	单链表、双链表、循环链表、静态链表	插入、删除灵活，不要求连续存储空间	不能随机访问，查找通常要顺序扫描

本章学习主线可以概括为：

- 线性表
- └ 逻辑结构：元素之间是一一对一的线性关系
 - └ 顺序存储：顺序表，用连续空间保存元素
 - └ 链式存储：链表，用指针或数组游标表示前后关系

复习重点：线性表是算法题命题重点。平时学习要特别重视基本操作的实现思想，尤其是顺序表与链表的插入、删除、查找、逆置、合并等操作。考试时通常不要求写出完全可运行的工程代码，更看重算法思想、关键步骤、边界条件和时间复杂度。

2.1 线性表的定义和基本操作

2.1.1 线性表的定义

线性表是由若干个**相同数据类型**的数据元素组成的**有限序列**。若表长为 0，则称为空表。若用 线性表名 L 表示一个线性表，它的一般形式为：

$$L = (a_1, a_2, \cdots, a_i, a_{i+1}, \cdots, a_n)$$

其中：

- $n \geq 0$ ， n 表示表长。
- a_1 是第一个元素，也称为**表头元素**。
- a_n 是最后一个元素，也称为**表尾元素**。
- 除第一个元素外，每个元素有且仅有一个直接前驱。
- 除最后一个元素外，每个元素有且仅有一个直接后继。

线性表的“线性”不是说元素一定按数值大小排列，而是说元素之间存在一种**一对一的逻辑关系**。也就是说，它强调的是元素之间的前后关系，而不是元素内容本身。

线性表的主要特点如下：

1. 元素个数有限

线性表不能无限长，表中元素数目为有限个。

2. 元素类型相同

表中所有元素属于同一种数据类型，每个元素在逻辑上是不可再分的整体。

3. 元素具有顺序性

元素之间存在明确的先后次序，除了首元素和尾元素外，每个元素都同时拥有直接前驱和直接后继。

4. 只讨论逻辑关系

线性表作为逻辑结构，只说明元素之间的相邻关系，不直接规定元素在内存中如何存放。

注意：线性表是一种逻辑结构，表示元素之间一对一的相邻关系；顺序表和链表是两种存储结构，属于实现层面的概念，不能混淆。

这一点很容易成为选择题陷阱：

说法	是否正确	理由
线性表中的元素一定连续存放	错	连续存放是顺序表的特点，不是线性表逻辑结构的要求
线性表中的元素都有直接前驱和直接后继	错	首元素没有直接前驱，尾元素没有直接后继
线性表中的元素类型相同	对	这是线性表定义的一部分
链表也是线性表的一种存储实现	对	链表用指针表示线性逻辑关系

2.1.2 线性表的基本操作

一个数据结构的基本操作，是围绕它最核心、最基础的功能抽象出来的操作集合。复杂算法往往由这些基本操作组合而成。

线性表的常见基本操作如下：

操作	含义	学习重点
InitList(&L)	初始化表，构造一个空线性表	注意引用或指针参数，操作会改变L本身
Length(L)	求表长，返回线性表L中元素个数	顺序表通常可直接返回长度变量
LocateElem(L, e)	按值查找，查找值为e的元素并返回位置	常与顺序扫描、时间复杂度结合考查
GetElem(L, i)	按位查找，获取第i个位置的元素值	顺序表可随机访问，链表通常要从头扫描

操作	含义	学习重点
<code>ListInsert(&L, i, e)</code>	插入操作，在第 i 个位置插入元素 e	重点掌握元素移动或指针修改
<code>ListDelete(&L, i, &e)</code>	删除操作，删除第 i 个位置元素，并用 e 返回其值	注意被删元素的保存、后续元素处理
<code>PrintList(L)</code>	输出操作，按前后顺序输出全部元素	通常作为辅助操作理解
<code>Empty(L)</code>	判空操作，判断线性表是否为空	空表时返回 <code>true</code> ，否则返回 <code>false</code>
<code>DestroyList(&L)</code>	销毁操作，释放线性表占用的存储空间	动态分配结构尤其要关注释放空间

注意：基本操作的具体实现取决于存储结构。相同的逻辑操作，在顺序表和链表中的实现方式、效率、边界处理都可能不同。符号 `&` 表示 C++ 中的引用调用；在 C 语言中，通常可通过指针实现类似效果。

学习这些操作时，要把握两层含义：

1. **逻辑层含义**：这个操作想完成什么任务。

例如 `ListInsert(&L, i, e)` 的目标是在逻辑序列的第 i 个位置插入 e 。

2. **存储层实现**：在具体存储结构中如何完成。

例如顺序表插入通常要移动元素；链表插入通常要修改指针。

考试中若要求写算法，不要只写函数名，要说明关键步骤。例如“删除第 i 个元素”至少要考虑：

- i 是否合法；
- 被删元素如何返回；
- 删除后表长如何变化；
- 顺序表是否需要移动元素；
- 链表是否需要找到被删结点的前驱。

2.2 线性表的顺序表示

2.2.1 顺序表的定义

考点追踪：顺序表的应用（2010、2011、2018、2020、2025）

线性表的顺序存储又称为**顺序表**。它使用一组地址连续的存储单元，按照线性表的逻辑顺序依次存储表中元素。

顺序表的核心特征是：**逻辑相邻，物理位置也相邻**。

若顺序表 L 的起始存储地址为 $LOC(A)$ ，每个元素占用的存储空间为 $sizeof(ElemType)$ ，则第 i 个元素 a_i 的存储地址为：

$$LOC(a_i) = LOC(A) + (i - 1) \times sizeof(ElemType)$$

如果按数组下标理解，则通常有如下对应关系：

线性表位序	数组下标	元素	存储地址
1	0	a_1	$LOC(A)$
2	1	a_2	$LOC(A) + sizeof(ElemType)$
i	$i - 1$	a_i	$LOC(A) + (i - 1) \times sizeof(ElemType)$
n	$n - 1$	a_n	$LOC(A) + (n - 1) \times sizeof(ElemType)$

注意：线性表中元素的位序通常从 1 开始，而数组下标通常从 0 开始。算法题中经常需要在“第 i 个元素”和“数组下标 i-1”之间转换。

由于顺序表中任意元素的地址都能由起始地址和下标直接计算出来，所以顺序表支持**随机访问**。访问任意位置元素的时间复杂度为 $O(1)$ 。

静态分配的顺序表常用数组实现：

```
#define MaxSize 50

typedef struct {
    ElemType data[MaxSize]; // 顺序表的元素
    int length;             // 顺序表的当前长度
} SqList;
```

这里需要区分两个变量：

名称	含义	是否变化
MaxSize	顺序表的最大容量	静态分配时通常固定
length	顺序表当前已有元素个数	随插入、删除动态变化

静态分配的问题是：数组大小在编译时已经确定，若空间占满后继续插入，可能出现溢出。为了提高灵活性，可以使用动态分配。

动态分配的顺序表可写成：

```
#define InitSize 100

typedef struct {
    ElemType *data; // 指向动态分配数组的指针
    int MaxSize, length; // 最大容量和当前长度
} SeqList;
```

C 语言中可以使用：

```
L.data = (ElemType *)malloc(sizeof(ElemType) * InitSize);
```

C++ 中可以使用：

```
L.data = new ElemType[InitSize];
```

注意：动态分配并不是链式存储，它仍然属于顺序存储结构。它的物理结构没有改变，仍然支持随机访问，只是存储空间大小可以在运行时动态调整。

顺序表的主要优点：

1. **支持随机访问**：通过首地址和元素序号可在 $O(1)$ 时间内找到指定元素。
2. **存储密度高**：每个结点只存储数据元素本身，不需要额外指针域。

顺序表的主要缺点：

1. **插入、删除效率较低**：通常需要移动大量元素。
2. **要求连续存储空间**：当连续空间不足时，即使总空闲内存足够，也可能无法分配成功。
3. **容量不够灵活**：静态分配容量固定；动态分配虽然能扩容，但扩容本身也需要复制元素。

从考研角度看，顺序表相关题目往往不会只考“定义”，而会把定义和操作结合起来，例如：给定一个数组形式的顺序表，要求删除重复元素、合并有序表、逆置线性表、移动区间元素等。因此，真正要掌握的是“连续存储 + 下标访问 + 元素移动”这三个关键词。

2.2.2 顺序表上基本操作的实现

考点追踪：顺序表上操作的时间复杂度分析（2023）

本小节只讨论顺序表中最核心的几类操作：初始化、插入、删除和按值查找。顺序表其他操作大多比较直接，例如求表长只需返回 `length`，按位查找只需通过数组下标访问。

学习顺序表操作时，不要把重点放在“代码能不能直接编译运行”上，而要抓住三件事：

1. **逻辑位序与数组下标的转换**：线性表第 i 个元素，对应数组下标 $i-1$ 。
2. **边界条件判断**：插入、删除前必须先判断位置是否合法，以及空间是否足够。
3. **移动方向**：插入要从后往前移动，删除要从前往后移动。

注意：在各种操作的实现中，通常可以忽略少量工程细节，例如变量声明、内存分配失败处理等。考试更看重算法核心思想和逻辑步骤。

1. 顺序表的初始化

静态分配的顺序表在声明时，数组空间已经分配好，因此初始化时通常只需要把当前长度设为 0。

```
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;

void InitList(SqList &L) {
```



```
L.length = 0;
}
```

动态分配的顺序表则需要在运行时申请初始数组空间，并记录当前容量上限。

```
#define InitSize 100

typedef struct {
    ElemType *data;
    int MaxSize, length;
} SeqList;

void InitList(SeqList &L) {
    L.data = (ElemType *)malloc(InitSize * sizeof(ElemType));
    L.length = 0;
    L.MaxSize = InitSize;
}
```

这里的 `MaxSize` 表示当前已分配空间的容量上限。当插入元素导致空间不足时，需要扩容。扩容的思想不是改变顺序存储的本质，而是重新申请一块更大的连续空间，再把原有元素复制过去。

分配方式	初始化重点	容量特点	是否仍是顺序存储
静态分配	令 <code>length = 0</code>	编译时固定	是
动态分配	申请数组空间，设置 <code>length</code> 和 <code>MaxSize</code>	运行时可调整	是

注意：动态分配并不是链式存储。只要元素仍然保存在连续地址空间中，并能通过下标随机访问，它就仍然是顺序存储。

2. 插入操作

插入操作是在顺序表 `L` 的第 `i` 个位置插入新元素 `e`。合法插入位置是：

$$1 \leq i \leq L.length + 1$$

其中，`i = 1` 表示在表头插入，`i = L.length + 1` 表示在表尾插入。若插入位置非法，或者当前表已满，则插入失败。

顺序表插入的关键是：**先给新元素腾出位置，再放入新元素**。由于第 `i` 个位置及其后的元素都要后移，所以移动时必须从最后一个元素开始向前移动，避免覆盖尚未移动的数据。

```
bool ListInsert(SqList &L, int i, ElemType e) {
    if (i < 1 || i > L.length + 1)
        return false;
    if (L.length >= MaxSize)
        return false;
```

```

    for (int j = L.length; j >= i; j--)
        L.data[j] = L.data[j - 1];

    L.data[i - 1] = e;
    L.length++;
    return true;
}

```

这段代码中最容易出错的是 `for` 循环的边界：

- `j = L.length`：从当前最后一个元素的后一个位置开始接收移动结果。
- `j >= i`：第 i 个元素及其后的元素都要后移。
- `L.data[j] = L.data[j - 1]`：体现“位序 i 对应下标 $i-1$ ”的转换。

插入示意可以用下面的数组变化理解：

原顺序表：

下标：	0	1	2	3	4	5	6	7
元素：	25	34	57	16	48	09	63	空

在第 4 个位置插入 50：

先将 16、48、09、63 依次后移一位，再把 50 放入下标 3。

插入后：

下标：	0	1	2	3	4	5	6	7
元素：	25	34	57	50	16	48	09	63

插入操作的时间复杂度分析如下：

情况	插入位置	元素移动次数	时间复杂度
最好情况	表尾插入, $i = n + 1$	0	$O(1)$
最坏情况	表头插入, $i = 1$	n	$O(n)$
平均情况	各位置等概率插入	$\frac{n}{2}$	$O(n)$

若在第 i 个位置插入元素，需要移动的元素个数为 $n - i + 1$ 。若每个合法插入位置概率相同，即 $p_i = \frac{1}{n + 1}$ ，则平均移动次数为：

$$\begin{aligned}
 \sum_{i=1}^{n+1} p_i (n - i + 1) &= \sum_{i=1}^{n+1} \frac{1}{n + 1} (n - i + 1) \\
 &= \frac{1}{n + 1} \sum_{i=1}^{n+1} (n - i + 1) \\
 &= \frac{1}{n + 1} \cdot \frac{n(n + 1)}{2} \\
 &= \frac{n}{2}
 \end{aligned}$$

因此，顺序表插入操作的平均时间复杂度为 $O(n)$ 。

3. 删除操作

删除操作是删除顺序表中第 i 个位置的元素，并通过参数 e 返回被删除元素的值。合法删除位置是：

$$1 \leq i \leq L.length$$

删除操作的关键是：**先保存被删除元素，再让后续元素前移补空缺。**

```
bool ListDelete(SqList &L, int i, ElemType &e) {
    if (i < 1 || i > L.length)
        return false;

    e = L.data[i - 1];

    for (int j = i; j < L.length; j++)
        L.data[j - 1] = L.data[j];

    L.length--;
    return true;
}
```

删除时，移动方向与插入相反：删除第 i 个元素后，第 $i + 1$ 个元素、第 $i + 2$ 个元素等依次向前移动一位。

删除示意如下：

原顺序表：

下标：

0

1

2

3

4

5

6

元素：

25

34

57

16

48

09

63

删除第 4 个位置的元素 16：

将 48、09、63 依次前移一位，表长减 1。

删除后：

下标：

0

1

2

3

4

5

元素：

25

34

57

48

09

63

删除操作的时间复杂度分析如下：

情况	删除位置	元素移动次数	时间复杂度
最好情况	删除表尾元素， $i = n$	0	$O(1)$
最坏情况	删除表头元素， $i = 1$	$n - 1$	$O(n)$
平均情况	各位置等概率删除	$\frac{n - 1}{2}$	$O(n)$

若删除第 i 个位置的元素，需要移动的元素个数为 $n - i$ 。若每个删除位置概率相同，即 $p_i = \frac{1}{n}$ ，则平均移动次数为：

$$\begin{aligned}\sum_{i=1}^n p_i(n-i) &= \sum_{i=1}^n \frac{1}{n}(n-i) \\ &= \frac{1}{n} \sum_{i=1}^n (n-i) \\ &= \frac{1}{n} \cdot \frac{n(n-1)}{2} \\ &= \frac{n-1}{2}\end{aligned}$$

因此，顺序表删除操作的平均时间复杂度为 $O(n)$ 。

4. 按值查找

按值查找也叫顺序查找，它是在顺序表中从前往后查找第一个值等于 e 的元素。若找到，返回该元素的位序；若找不到，返回 0。

```
int LocateElem(SqList L, ElemType e) {
    int i;
    for (i = 0; i < L.length; i++)
        if (L.data[i] == e)
            return i + 1;
    return 0;
}
```

这里返回 `i + 1`，不是返回 `i`，因为数组下标从 0 开始，而线性表位序从 1 开始。返回 0 的好处是：0 不属于合法位序，可以用来表示查找失败。

按值查找的时间复杂度分析如下：

情况	说明	比较次数	时间复杂度
最好情况	目标元素在表头， $i = 1$	1	$O(1)$
最坏情况	目标元素在表尾或不存在	n	$O(n)$
平均情况	目标元素在各位置概率相同	$\frac{n+1}{2}$	$O(n)$

若目标元素出现在第 i 个位置的概率为 $p_i = \frac{1}{n}$ ，则平均比较次数为：

$$\begin{aligned}\sum_{i=1}^n p_i \cdot i &= \sum_{i=1}^n \frac{1}{n} i \\ &= \frac{1}{n} \cdot \frac{n(n+1)}{2} \\ &= \frac{n+1}{2}\end{aligned}$$

因此，顺序表按值查找的平均时间复杂度为 $O(n)$ 。

需要特别区分“按值查找”和“按序号查找”：

操作	含义	顺序表时间复杂度	原因
按值查找	给定元素值，找它在哪里	$O(n)$	需要从前往后比较
按序号查找	给定位序 i ，取第 i 个元素	$O(1)$	可直接通过数组下标访问

这也是顺序表最重要的性质之一：**它支持随机访问，但不代表所有查找都是 $O(1)$** 。只有在已知位置或下标时，顺序表才能直接定位；若只知道元素值，仍然需要顺序扫描。

本小节可以用下面的表快速收束：

操作	核心动作	最好	最坏	平均	关键原因
初始化	置空表，必要时申请空间	$O(1)$	$O(1)$	$O(1)$	只改少量变量
插入	后移元素，放入新值	$O(1)$	$O(n)$	$O(n)$	可能移动大量元素
删除	保存被删值，前移元素	$O(1)$	$O(n)$	$O(n)$	可能移动大量元素
按值查找	从前往后比较	$O(1)$	$O(n)$	$O(n)$	不能由值直接算位置
按序号查找	通过下标访问	$O(1)$	$O(1)$	$O(1)$	顺序存储支持随机访问

从考试角度看，顺序表算法题往往围绕“移动元素”展开。判断一个顺序表算法复杂度时，重点看是否存在成段移动或成段扫描，而不是看代码里有没有数组。

2.3 线性表的链式表示

顺序表的核心特点是：元素放在一段连续存储空间中，因此可以随机访问任意元素。但它的缺点也很明显：插入和删除时往往要移动大量元素。

链式存储的思路正好相反：**不要求物理地址连续，而是用指针把逻辑上相邻的元素连接起来**。也就是说，元素在内存中可以分散存放，只要每个结点能指出下一个结点的位置，就能维持线性表的前后关系。

可以把顺序表和链表的差别理解为：

对比角度	顺序表	链表
存储空间	要求连续空间	不要求连续空间
逻辑关系表示方式	由数组下标天然体现	由指针显式连接
随机访问	支持，按位序访问为 $O(1)$	不支持，按位序访问通常为 $O(n)$
插入删除	可能移动大量元素	通常只需修改相关指针
额外开销	主要存数据本身	每个结点还要存指针域

链式存储牺牲了顺序表的随机访问能力，换来了更灵活的插入、删除能力。本节的核心任务，就是掌握链表如何通过指针建立逻辑关系，以及各种基本操作中指针应该怎样修改。

2.3.1 单链表的定义

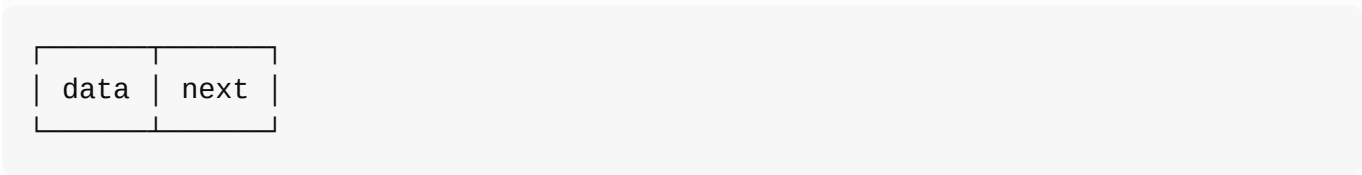
考点追踪：单链表的应用（2009、2012、2013、2015、2016、2019）

单链表是线性表的一种链式存储结构。它用一组任意的存储单元存放线性表中的元素，存储单元之间不要求地址连续。

为了表示元素之间的前后关系，单链表中的每个结点通常包含两个部分：

组成部分	名称	作用
data	数据域	存放数据元素本身
next	指针域	存放后继结点的地址

结点结构可以写成：



在 C 语言风格的算法描述中，单链表结点类型常定义如下：

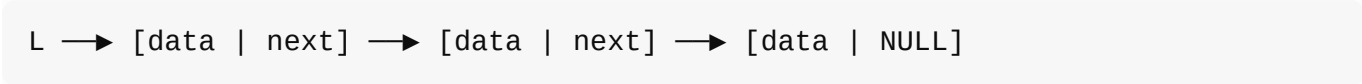
```
typedef struct LNode {
    ElemType data;           // 数据域
```

```
struct LNode *next;    // 指针域
} LNode, *LinkList;
```

这里有两个名字要分清：

- `LNode` 表示一个结点类型。
- `LinkList` 表示指向 `LNode` 的指针类型，常用于表示整个单链表的头指针。

例如，`LinkList L` 本质上等价于 `LNode *L`，它不是一个完整结点，而是指向某个结点的指针。单链表依赖 `next` 指针把结点连起来。若某结点是最后一个结点，则它没有后继，通常令其 `next` 为 `NULL`。



单链表常见两种形式：

类型	结构特点	空表判断	表头插删是否特殊
带头结点单链表	第一个结点是头结点，通常不存实际数据	<code>L->next == NULL</code>	不特殊
不带头结点单链表	头指针直接指向第一个数据结点	<code>L == NULL</code>	通常需要特殊处理

这里必须区分**头指针**和**头结点**：

- **头指针**是指向链表起始位置的指针变量，单链表必须有头指针，否则无法找到整个链表。
- **头结点**是链表中附加在第一个数据结点之前的特殊结点，它可以没有实际数据，只是为了统一操作。

带头结点单链表可以表示为：



不带头结点单链表可以表示为：



教材中特别强调：无论是否带头结点，**头指针始终指向链表的第一个结点**。只不过在带头结点的单链表中，第一个结点是头结点；在不带头结点的单链表中，第一个结点就是第一个数据结点。

引入头结点的主要优点有两个：

1. 操作统一

第一个数据结点的位置存放在头结点的指针域中，因此在表头插入、删除时，与其他位置的操作逻辑一致，不必单独处理头指针。

2. 空表与非空表处理统一

头指针始终指向头结点，空表只表现为头结点的 `next` 为 `NULL`，避免了对头指针本身是否为空的

频繁判断。

从考试角度看，带头结点是算法题中更常见的默认设定。若题目没有特殊说明，很多教材和真题默认使用带头结点的单链表；但做题时仍要看清题干，因为“不带头结点”时表头插入、表头删除会明显不同。

2.3.2 单链表上基本操作的实现

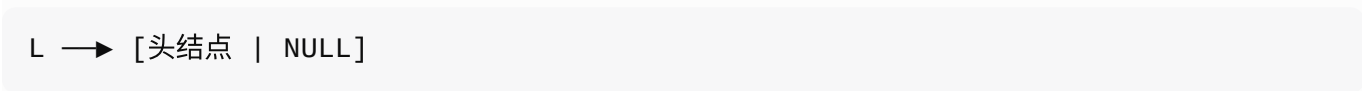
带头结点的单链表在操作实现上更方便。因此，若无特殊说明，本节算法默认链表带头结点。

1. 单链表的初始化

带头结点的单链表初始化时，需要创建一个头结点，并令头指针 `L` 指向该头结点，再把头结点的 `next` 初始化为 `NULL`。

```
bool InitList(LinkList &L) {
    L = (LNode *)malloc(sizeof(LNode));
    L->next = NULL;
    return true;
}
```

初始化完成后的空表为：



不带头结点的单链表初始化更简单，只需令头指针为空：

```
bool InitList(LinkList &L) {
    L = NULL;
    return true;
}
```

两者的核心差别在于：

类型	初始化结果	空表判断
带头结点	<code>L</code> 指向头结点	<code>L->next == NULL</code>
不带头结点	<code>L == NULL</code>	<code>L == NULL</code>

需要注意 C 语言结构体指针的访问方式。设 `p` 是指向结点的指针，则：

写法	含义	等价写法
<code>p->data</code>	访问 <code>p</code> 所指结点的数据域	<code>(*p).data</code>
<code>p->next</code>	访问 <code>p</code> 所指结点的指针域	<code>(*p).next</code>
<code>p->next->data</code>	访问后继结点的数据域	<code>*(p->next).data</code>

也就是说，`. 左侧`应是结构体变量，`->` 左侧应是结构体指针。链表算法中的很多错误，本质上都是没有分清“指针变量”和“指针所指结点”。

2. 求表长操作

单链表不能像顺序表那样直接用 `length` 变量得到表长，除非结构中额外维护了长度字段。一般情况下，需要从第一个数据结点开始，沿 `next` 逐个遍历并计数。

```
int Length(LinkList L) {
    int len = 0;
    LNode *p = L;
    while (p->next != NULL) {
        p = p->next;
        len++;
    }
    return len;
}
```

这段代码默认链表带头结点，因此 `p` 初始指向头结点，每向后访问一个数据结点，计数加 1。头结点不计入表长。

求表长需要访问每个数据结点一次，因此时间复杂度为 $O(n)$ 。

操作	顺序表	单链表
获取表长	通常直接读 <code>length</code> ， $O(1)$	通常遍历计数， $O(n)$

这说明链表并不是所有操作都比顺序表快。链表的优势主要体现在插入、删除时避免大量移动元素，而不是访问和统计。

3. 按序号查找结点

按序号查找是给定位置 `i`，返回第 `i` 个结点的指针。单链表不能直接按下标定位，只能从头开始沿 `next` 指针逐个查找。

```
LNode *GetElem(LinkList L, int i) {
    LNode *p = L;
    int j = 0;
    while (p != NULL && j < i) {
        p = p->next;
        j++;
    }
    return p;
}
```

这段代码把头结点视为第 0 个结点。若要找第 1 个数据结点，就从头结点向后走一步；若要找第 `i` 个数据结点，就向后走 `i` 步。若 `i` 超过表长，则最终返回 `NULL`。

按序号查找的时间复杂度为 $O(n)$ 。

顺序表和单链表在“按序号查找”上的差异非常重要：

操作	顺序表	单链表
查找第 i 个元素	可通过下标直接定位， $O(1)$	只能从头扫描， $O(n)$

这也是“顺序表支持随机访问，链表不支持随机访问”的最直接体现。

4. 按值查找表结点

按值查找是从第一个数据结点开始，依次比较数据域，若某结点的数据域等于给定值 e ，则返回该结点指针；否则返回 `NULL`。

```
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e)
        p = p->next;
    return p;
}
```

按值查找仍然要顺序扫描，时间复杂度为 $O(n)$ 。

需要区分两种查找：

查找方式	已知条件	单链表做法	时间复杂度
按序号查找	知道第几个	从头结点开始走 i 步	$O(n)$
按值查找	知道元素值	从第一个数据结点开始逐个比较	$O(n)$

顺序表中“按序号查找”为 $O(1)$ ，但单链表中“按序号查找”和“按值查找”一般都是 $O(n)$ 。

5. 插入结点操作

考点追踪：单链表插入操作的过程（2016、2024）

将值为 e 的新结点插入到第 i 个位置，含义是让新结点成为第 i 个数据结点。对于带头结点的单链表，基本思路如下：

- 1. 检查并寻找第 $i - 1$ 个结点，也就是插入位置的前驱结点。
- 2. 创建新结点 `s`，把 e 存入 `s->data`。
- 3. 令 `s->next` 指向原来第 i 个结点。
- 4. 令前驱结点的 `next` 指向新结点 `s`。

代码如下：

```
bool ListInsert(LinkList &L, int i, ElemType e) {
    LNode *p = L;
    int j = 0;
    while (p != NULL && j < i - 1) {
        p = p->next;
    }
```

```

        j++;
    }
    if (p == NULL)
        return false;

    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next = p->next;
    p->next = s;
    return true;
}

```

插入时最关键的两句是：

```

s->next = p->next;
p->next = s;

```

它们的顺序不能颠倒。若先执行 `p->next = s`，原来第 i 个结点的地址就可能丢失，无法再令 `s->next` 指向原后继结点，甚至可能把链表错误地接成自环。

插入示意可以写成：

原链：p → b

先让 s 接上原后继：

s → b

再让 p 指向 s：

p → s → b

时间复杂度要分情况看：

情况	关键耗时	时间复杂度
只知道插入位置 i	需要从头找第 $i - 1$ 个结点	$O(n)$
已知前驱结点 p	只需修改两条指针	$O(1)$

在带头结点单链表中，若插入位置为 `i == 1`，也就是在第一个数据结点之前插入，只需把头结点当作前驱结点即可，不需要特殊处理。若链表不带头结点，表头插入往往要修改头指针 `L`，需要额外判断。

扩展：对某个结点进行前插操作

所谓前插，是在某个目标结点 `p` 之前插入新结点。单链表没有前驱指针，因此常规做法是从头开始找 `p` 的前驱，找到后再做后插，时间复杂度为 $O(n)$ 。

若题目已经给定目标结点 `p`，并且允许修改结点数据域，可以使用一个常见技巧：**先在 `p` 后面插入新结点 `s`，再交换 `p` 与 `s` 的数据域**。这样逻辑效果等价于在 `p` 前插入，且不需要寻找前驱。

关键代码为：

```
s->next = p->next;
p->next = s;
temp = p->data;
p->data = s->data;
s->data = temp;
```

这个技巧的本质是“结点位置没有前插，但数据效果实现了前插”。它适合数据域可复制、可交换的情况，时间复杂度为 $O(1)$ 。

6. 删除结点操作

删除单链表的第 i 个数据结点时，同样需要先找到第 $i - 1$ 个结点，也就是被删结点的前驱。设前驱结点为 p ，被删结点为 q ，则操作顺序是：

1. 令 $q = p->next$ ，保存被删结点地址。
2. 用 $e = q->data$ 保存被删元素的值。
3. 令 $p->next = q->next$ ，把 q 从链表中断开。
4. $free(q)$ ，释放被删结点空间。

代码如下：

```
bool ListDelete(LinkList &L, int i, ElemType &e) {
    LNode *p = L;
    int j = 0;
    while (p->next != NULL && j < i - 1) {
        p = p->next;
        j++;
    }
    if (p->next == NULL || j > i - 1)
        return false;

    LNode *q = p->next;
    e = q->data;
    p->next = q->next;
    free(q);
    return true;
}
```

删除示意如下：

原链： $p \rightarrow q \rightarrow c$

跳过 q ：

$p \rightarrow c$

```
释放 q:
free(q)
```

类似插入操作，删除操作的主要耗时也在查找前驱结点上，因此时间复杂度为 $O(n)$ 。若已经给定被删结点的前驱 p ，则删除只需改指针和释放空间，时间复杂度为 $O(1)$ 。

带头结点单链表删除首元结点时，与删除其他结点的逻辑相同，因为头结点就是首元结点的前驱；不带头结点单链表删除首元结点时，需要额外修改头指针 L 。

扩展：删除指定结点 p

若要删除某个给定结点 p ，常规做法仍然是从头开始查找它的前驱，时间复杂度为 $O(n)$ 。

如果允许修改结点内容，并且 p 不是尾结点，可以使用 $O(1)$ 技巧：把 p 的后继结点的数据复制到 p 中，然后删除 p 的后继结点。关键代码如下：

```
LNode *q = p->next;
p->data = p->next->data;
p->next = q->next;
free(q);
```

这个方法看起来删除的是 p 的后继结点，实际效果却是让 p 原来的数据消失，因此逻辑上相当于删除了 p 。但它有一个重要限制：**不能用于删除尾结点**，因为尾结点没有后继结点可供复制和释放。

到这里，单链表插入与删除的核心规律可以总结为：

操作	是否需要前驱	若只给位 序	若已知前驱	关键动作
插入第 i 个 位置	需要第 $i - 1$ 个结点	$O(n)$	$O(1)$	<code>s->next = p->next; p->next = s;</code>
删除第 i 个 位置	需要第 $i - 1$ 个结点	$O(n)$	$O(1)$	<code>q = p->next; p->next = q->next; free(q);</code>
给定结点 前插	普通做法需要 前驱	$O(n)$	可用后插加换数据做到 $O(1)$	后插新结点，再交换数据域
删除给定 结点	普通做法需要 前驱	$O(n)$	若非尾结点可复制后继 做到 $O(1)$	复制后继数据，再删除后继

单链表算法题的高频陷阱是“只改一个指针”。无论插入还是删除，都要先想清楚会不会丢失后继地址、会不会形成断链或自环、是否需要释放空间、头结点是否参与计数。**7. 采用头插法建立单链表** 头插法的思想是：每读入一个新元素，就创建一个新结点，并把它插入到当前链表的最前端。由于新结点总是插入到头结点之后，所以它会逐渐成为新的首元结点。

头插法的逻辑过程可以理解为：

初始: $L \rightarrow \text{NULL}$

读入 a1: $L \rightarrow a1$

读入 a2: $L \rightarrow a2 \rightarrow a1$

读入 a3: $L \rightarrow a3 \rightarrow a2 \rightarrow a1$

因此，**头插法建立出来的链表元素顺序与输入顺序相反**。这一点在算法题中非常重要：头插法常被用来实现链表逆置，或者在边读入边构造“逆序链表”时使用。

典型代码如下：

```
LinkList List_HeadInsert(LinkList &L) {
    LNode *s;
    int x;
    L = (LNode *)malloc(sizeof(LNode));
    L->next = NULL;

    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = L->next;
        L->next = s;
        scanf("%d", &x);
    }
    return L;
}
```

其中 9999 只是输入结束标志，不是链表元素本身。核心语句仍然是两条指针修改：

```
s->next = L->next;
L->next = s;
```

这两句表示：先让新结点 **s** 指向原来的首元结点，再让头结点指向 **s**。顺序不能反，否则会丢失原首元结点。

每插入一个结点只需修改常数条指针，单次插入时间复杂度为 $O(1)$ 。若共插入 n 个结点，则总时间复杂度为 $O(n)$ 。

需要注意的是，以上代码默认**带头结点**。若单链表不带头结点，则每次在表头插入新结点后，都要把头指针 **L** 更新为新结点地址，否则链表入口不会指向新的首元结点。

8. 采用尾插法建立单链表

尾插法的思想是：每读入一个新元素，就把新结点插入到当前链表的表尾。为了避免每次都从头遍历到尾部，需要设置一个**尾指针 r**，始终指向当前链表的尾结点。

尾插法的逻辑过程如下：

初始: $L \rightarrow \text{NULL}$, r 指向头结点 L

读入 a_1 : $L \rightarrow a_1$, r 指向 a_1

读入 a_2 : $L \rightarrow a_1 \rightarrow a_2$, r 指向 a_2

读入 a_3 : $L \rightarrow a_1 \rightarrow a_2 \rightarrow a_3$, r 指向 a_3

因此，尾插法建立出来的链表元素顺序与输入顺序一致。

典型代码如下：

```
LinkList List_TailInsert(LinkList &L) {
    int x;
    L = (LNode *)malloc(sizeof(LNode));
    LNode *s, *r = L;

    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        r->next = s;
        r = s;
        scanf("%d", &x);
    }
    r->next = NULL;
    return L;
}
```

尾插法的关键是维护 r ：

```
r->next = s;
r = s;
```

第一句把新结点接到原尾结点之后，第二句把尾指针更新到新结点。循环结束后，需要执行：

```
r->next = NULL;
```

这表示最后一个结点没有后继，链表正式结束。

尾插法由于维护了尾指针，不需要每次遍历寻找尾部，因此每次插入的时间复杂度为 $O(1)$ ，建立 n 个结点的总时间复杂度为 $O(n)$ 。

头插法与尾插法的区别可以总结为：

建表方法	新结点插入位置	元素顺序	是否常用尾指针	总时间复杂度	常见用途
头插法	头结点之后	与输入顺序相反	否	$O(n)$	逆置链表、反序建表

建表方法	新结点插入位置	元素顺序	是否常用尾指针	总时间复杂度	常见用途
尾插法	当前表尾之后	与输入顺序一致	是	$O(n)$	正序建表、保持输入顺序

注意：单链表是链式存储结构的基础。设计算法时，建议先画图理清指针变化逻辑，再写代码。常见错误包括指针丢失、断链、形成自环、忘记释放结点空间等。

2.3.3 双链表

单链表的每个结点只有一个 `next` 指针，只能从前往后遍历。若要访问某结点的前驱，通常必须从头结点重新扫描，时间复杂度为 $O(n)$ 。为了弥补这一不足，可以使用**双链表**。

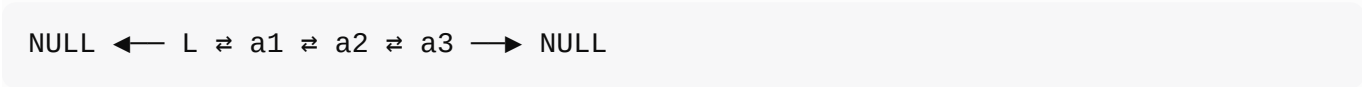
双链表的每个结点除数据域外，还包含两个指针域：

指针域	含义
<code>prior</code>	指向直接前驱结点
<code>next</code>	指向直接后继结点

结点结构可定义为：

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinklist;
```

带头结点双链表的典型逻辑形态为：



其中：

- 头结点的 `prior` 通常为 `NULL`；
- 尾结点的 `next` 通常为 `NULL`；
- 中间每个结点既能通过 `next` 找后继，也能通过 `prior` 找前驱。

双链表的按位查找、按值查找与单链表类似，通常仍需要从头开始顺序扫描。但在**已知某个目标结点**的前提下，双链表能够直接访问它的前驱和后继，因此插入、删除操作可以做到 $O(1)$ 。

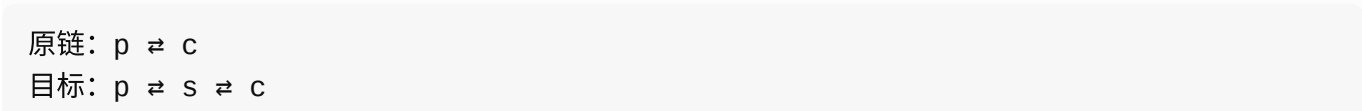
1. 双链表的插入操作

考点追踪：双链表中插入操作的实现（2023）

若要在双链表结点 `p` 之后插入新结点 `s`，需要同时维护前驱方向和后继方向的链接。核心步骤如下：

```
s->next = p->next;
p->next->prior = s;
s->prior = p;
p->next = s;
```

对应关系可以理解为：



1. s 先指向 c
2. c 的 prior 改为 s
3. s 的 prior 改为 p
4. p 的 next 改为 s

其中最容易出错的是执行顺序。`s->next = p->next` 必须在 `p->next = s` 之前完成，否则原后继结点地址会被覆盖，导致无法再连接原后继结点。

若 p 是尾结点，即 `p->next == NULL`，则不存在原后继结点，此时不能执行 `p->next->prior = s`，需要单独判断。这类边界条件在写完整算法时不能遗漏。

如果题目要求在结点 p 之前插入新结点 s，由于双链表能通过 `p->prior` 直接找到前驱，所以可转化为“在 `p->prior` 之后插入 s”。这正是双链表相对单链表的重要优势。

2. 双链表的删除操作

考点追踪：双链表中删除操作的实现（2016）

若要删除双链表中结点 p 的后继结点 q，需要让 p 越过 q 直接连到 q 的后继，同时让 q 的后继反向连回 p。

典型代码如下：

```
p->next = q->next;
q->next->prior = p;
free(q);
```

对应关系为：

原链：p ⇌ q ⇌ c
目标：p ⇌ c，然后释放 q

如果 q 是尾结点，则 `q->next == NULL`，不存在 `q->next->prior`，也需要单独判断。

双链表删除的本质是同时维护两条链：

1. 后继方向：前驱结点的 `next` 要跳过被删结点；
2. 前驱方向：后继结点的 `prior` 要跳过被删结点。

单链表与双链表的关键差异如下：

操作场景	单链表	双链表
已知结点，找后继	$O(1)$	$O(1)$
已知结点，找前驱	通常 $O(n)$	$O(1)$
已知目标结点后插	$O(1)$	$O(1)$ ，但要改更多指针

操作场景	单链表	双链表
已知目标结点前插	通常 $O(n)$ ；可用换数据技巧	$O(1)$
删除给定非尾结点	可用复制后继技巧做到 $O(1)$	通常可直接 $O(1)$

双链表的优势是访问前驱方便，缺点是每个结点需要额外的 `prior` 指针，存储开销更大，指针维护也更复杂。

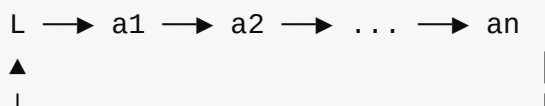
2.3.4 循环链表

循环链表的核心特点是：链表最后一个结点不再指向 `NULL`，而是指回链表中的某个结点，从而形成一个环。循环结构使得从某个结点出发，可以不断沿指针访问并回到起点。

1. 循环单链表

循环单链表与普通单链表的主要区别在于：尾结点的 `next` 域不再为 `NULL`，而是指向头结点。

带头结点循环单链表可以表示为：



由于不存在 `next == NULL` 的结点，因此循环单链表的判空条件不再是 `L == NULL` 或 `L->next == NULL`，而是：

```
L->next == L
```

这里默认 `L` 是头结点。空表时，头结点的 `next` 指向它自己。

考点追踪：循环单链表中删除首元素的操作（2021）

循环单链表的插入、删除操作与普通单链表基本类似，但在表尾操作时必须维护“尾结点指回头结点”的循环关系。例如，在尾结点 `r` 之后插入新结点 `s` 时，可执行：

```
s->next = r->next;
r->next = s;
r = s;
```

其中 `r->next` 原本指向头结点，因此 `s->next = r->next` 能让新结点继续指回头结点，再把 `r->next` 改为 `s`，最后更新尾指针。

循环单链表有时不设头指针，而只设尾指针 `r`。这样做的好处是：

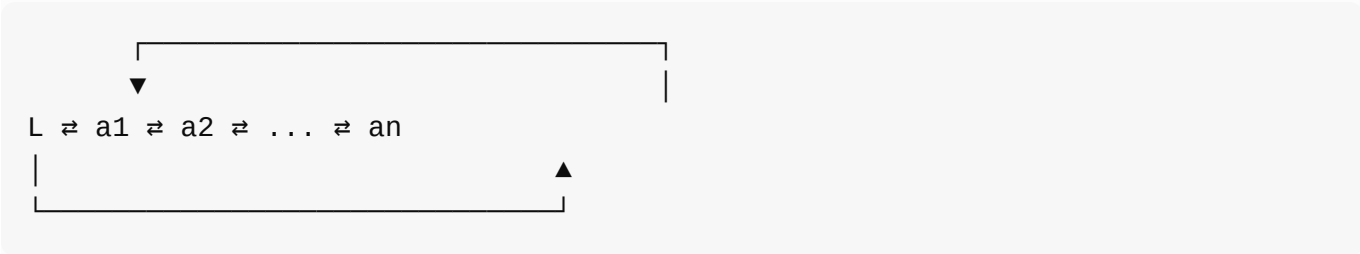
- 访问表尾：直接通过 `r` 完成，时间复杂度为 $O(1)$ ；
- 访问表头：通过 `r->next` 完成，时间复杂度为 $O(1)$ 。

若只使用头指针，要在表尾插入新结点通常需要从头遍历到尾，时间复杂度为 $O(n)$ 。因此，循环单链表配合尾指针时，常能提高表头、表尾操作效率。

2. 循环双链表

循环双链表可以看作“双链表 + 循环结构”。它不仅让尾结点的 `next` 指向头结点，还让头结点的 `prior` 指向尾结点。

带头结点循环双链表可以表示为：



设尾结点为 `p`，则有：

```
p->next == L;
L->prior == p;
```

当循环双链表为空时，头结点的两个指针都指向自身：

```
L->next == L;
L->prior == L;
```

循环双链表的优势是能够从任意方向循环移动，特别适合需要频繁在两端操作、或需要循环遍历的场景。但它的指针维护最复杂：既要保持前后双向连接，又要保持首尾成环。

2.3.5 静态链表

静态链表是用数组来模拟链式存储的结构。它不使用真实指针，而是用数组下标表示“下一个结点在哪里”，这个下标也称为**游标**。

静态链表中每个结点通常包含两个域：

域	含义
<code>data</code>	存储数据元素
<code>next</code>	存储下一个元素所在的数组下标

结构定义如下：

```
#define MaxSize 50

typedef struct {
    ElemType data;
```

```
int next;
} SLinkList[MaxSize];
```

例如，若数组中有如下关系：

数组下标	data	next
2	a	1
1	b	6
6	c	3
3	d	-1

则链表的逻辑顺序为：

a → b → c → d

其中 `next == -1` 表示链表结束。

静态链表的特点如下：

- 1. **逻辑上是链式结构**：元素的先后关系由 `next` 游标维护，而不是由数组下标自然决定。
- 2. **物理上借助数组实现**：需要预先分配一块连续数组空间。
- 3. **插入、删除不需要移动大量元素**：只需修改 `next` 游标，类似普通链表修改指针。
- 4. **容量不够灵活**：数组大小固定，空间不足时仍可能溢出。
- 5. **存储密度介于顺序表和动态链表之间**：每个结点除数据域外，还要额外存储 `next` 下标。

静态链表常用于不支持指针的语言或场景中，例如一些早期语言环境。它的思想非常适合帮助理解“链式存储的本质不是指针本身，而是通过某种方式表示后继关系”。

2.3.6 顺序表和链表的比较

顺序表和链表都能表示线性表，但它们适合的场景不同。比较时不能只说“谁更好”，而要从存取方式、逻辑结构与物理结构、查找插入删除、空间分配和使用环境等角度综合判断。

1. 存取方式

顺序表支持随机存取。因为元素连续存放，可以通过首地址和下标直接计算任意元素地址，所以访问第 i 个元素的时间复杂度为 $O(1)$ 。

链表只支持顺序存取。若要访问第 i 个元素，必须从头结点开始沿指针逐个向后遍历，平均时间复杂度为 $O(n)$ 。

操作	顺序表	链表
访问第 i 个元素	$O(1)$	$O(n)$
从前到后遍历全部元素	$O(n)$	$O(n)$

因此，只要题目强调“频繁按序号访问”“随机访问”“快速读取第 i 个元素”，通常更偏向顺序表。

2. 逻辑结构与物理结构

顺序表中，逻辑上相邻的元素在物理内存中也相邻，元素之间的前后关系由存储地址自然体现。
链表中，逻辑上相邻的元素在物理内存中不一定相邻，元素之间的关系依靠指针或游标维护。

结构	逻辑相邻	物理相邻	关系维护方式
顺序表	是	是	元素位置和下标
链表	是	不一定	指针或游标

这也是两类结构本质差异的来源：顺序表牺牲插入删除灵活性，换取随机访问能力；链表牺牲随机访问能力，换取插入删除时的灵活连接。

3. 查找、插入和删除操作

对于按值查找，如果线性表无序，顺序表和链表都只能顺序扫描，时间复杂度均为 $O(n)$ 。若线性表有序，顺序表可以使用折半查找，时间复杂度为 $O(\log_2 n)$ ；链表由于不能随机访问中间元素，通常仍需顺序查找，时间复杂度为 $O(n)$ 。

对于按序号查找，顺序表可直接通过下标访问，时间复杂度为 $O(1)$ ；链表需要从头扫描，时间复杂度为 $O(n)$ 。

对于插入和删除，顺序表需要移动元素。若在等概率位置进行插入或删除，平均大约要移动一半元素，因此时间复杂度为 $O(n)$ 。链表在已知操作位置或前驱结点时，只需修改指针，时间复杂度为 $O(1)$ ；但如果还需要先定位目标位置，则定位本身通常为 $O(n)$ 。

操作	顺序表	链表	关键原因
无序按值查找	$O(n)$	$O(n)$	都要顺序扫描
有序按值查找	可折半, $O(\log_2 n)$	通常 $O(n)$	链表不支持随机访问
按序号查找	$O(1)$	$O(n)$	顺序表可直接下标定位
已知位置插入	平均 $O(n)$	修改指针 $O(1)$	顺序表要移动元素
先定位再插入	通常 $O(n)$	通常 $O(n)$	链表定位要遍历
已知位置删除	平均 $O(n)$	修改指针 $O(1)$	顺序表要移动元素
先定位再删除	通常 $O(n)$	通常 $O(n)$	链表定位要遍历

这里有一个高频误区：**不能简单说链表插入、删除一定比顺序表快**。如果题目没有给出待插入或待删除位置的指针，链表仍然需要先遍历定位，整体时间复杂度仍可能是 $O(n)$ 。

4. 空间分配

顺序表通常需要预先分配连续空间。若容量设得过大，会造成内存浪费；若容量设得过小，又容易在插入时溢出。动态分配顺序表虽然支持扩容，但扩容时往往需要申请新的连续空间，并复制原有元素，代价较高。

链表采用动态结点分配，按需申请空间，不要求整块连续内存，扩展更灵活。但每个结点都需要额外指针域，因此存储密度小于 1，空间利用率相对较低。

比较角度	顺序表	链表
是否要求连续空间	要求	不要求
是否可按需扩展	静态表不灵活；动态表需扩容复制	灵活，按需申请结点
额外存储开销	通常无额外指针域	每个结点需要指针域
存储密度	高	较低

5. 如何选择存储结构

实际选择顺序表还是链表，可以从以下三个角度判断：

1. 基于存储的考虑

若线性表长度或规模难以预估，顺序表因需要预先分配固定容量而不适合；链表无需预设固定容量，可按需动态扩展，但每个结点会带来额外指针开销。

2. 基于运算的考虑

若应用中频繁按序号访问元素，顺序表的 $O(1)$ 随机访问明显更高效。若应用中插入、删除操作非常频繁，并且常能直接定位到操作位置，链表通常更合适。

3. 基于环境的考虑

顺序表基于数组实现，几乎所有高级语言都支持，实现简单；链表依赖指针或引用操作，在部分语言或运行环境中实现更复杂。因此，开发语言和运行环境也会影响结构选择。

综合来看：

场景	更适合的结构	理由
长度基本固定，主要按序号访问	顺序表	随机访问快，存储密度高
长度变化大，频繁插入删除	链表	扩展灵活，改指针即可完成连接
有序表需要折半查找	顺序表	链表无法高效随机访问中间元素
内存碎片较多，难以获得连续大块空间	链表	不要求连续存储
每个元素很小且数量巨大	顺序表	链表指针域开销相对更明显

注意：只有熟练掌握线性表的顺序存储和链式存储，才能深刻理解它们的优缺点。算法题中，选择结构时不要背结论，而要结合“访问多还是插删多”“是否需要随机访问”“是否能提前定位操作位置”“空间是否连续”等条件分析。

归纳总结

第二章算法题的核心考法

本章是算法设计题的重要考查内容。线性表相关算法题通常代码量不大，但很容易在**边界条件、指针修改顺序、时间复杂度分析**上失分。复习时要把本章视为“算法题基本功训练章”，而不是只背概念。

从题目要求看，线性表算法题通常围绕下面三件事展开：

1. 给出算法设计思想

先说明准备用什么结构、从哪里开始扫描、遇到目标元素如何处理、是否需要辅助空间。算法思想是答题的骨架。

2. 用 C 或 C++ 风格伪代码描述关键步骤

代码不一定追求工程完整，但关键指针、循环条件、赋值顺序必须准确。涉及链表时，最好写清楚结点指针的变化；涉及顺序表时，最好写清楚下标范围和移动方向。

3. 分析时间复杂度和空间复杂度

复杂度分析不能只写结论，要能说清主要开销来自哪里。例如顺序表删除元素的主要代价是移动元素，链表按值查找的主要代价是顺序遍历。

注意：在算法设计题中，若能正确定义数据结构并清晰阐述算法思想，通常可以获得至少一半的分数；若能进一步用规范代码实现，则得分更有保障；逻辑较复杂的部分可直接用文字说明，确保思路完整。

常见算法设计技巧

本章算法题虽然形式多，但常用技巧相对集中。可以按存储结构分为“顺序表技巧”和“链表技巧”。

对象	常用技巧	适用场景	核心关注点
顺序表	双指针扫描	删除、去重、划分、合并	下标移动方向与元素覆盖关系
顺序表	归并	两个有序表合并、求中位数	谁小先放谁，剩余部分继续复制
顺序表	二分查找	有序顺序表查找、定位插入点	必须支持随机访问
链表	头插法	逆置链表、建立逆序链表	新结点或原结点插入到头结点之后
链表	尾插法	保持输入顺序建立链表	需要维护尾指针
链表	逆置法	单链表就地逆置	断链前保存后继结点
链表	快慢指针	找中点、判断环、寻找倒数第 k 个结点	快指针走得更快，慢指针滞后定位
链表	双指针法	删除指定结点、分离两个链表	常用前驱指针配合当前指针

顺序表算法更像“数组下标控制题”。做题时应重点检查：

- 循环从左向右还是从右向左。
- 覆盖元素时是否会覆盖尚未处理的数据。
- 插入或删除后表长是否及时更新。

- 下标是否合法，尤其是第一个元素和最后一个元素。

链表算法更像“指针连接题”。做题时应重点检查：

- 是否需要头结点，头结点是否参与数据处理。
- 删除结点前是否保存了前驱结点。
- 改变指针前是否保存了后继结点。
- 最后一个结点的 `next` 是否置空。
- 空表、单结点表、只有两个结点的表是否能正确处理。

典型算法题答题框架

线性表算法题可以采用“三段式”组织答案：

第一段：算法思想

说明使用顺序表还是链表，如何扫描、如何判断、如何修改结构。

第二段：算法代码

写出关键伪代码或 C/C++ 风格代码，重点保证循环、指针、下标正确。

第三段：复杂度分析

说明时间复杂度和空间复杂度，并指出主要开销来源。

例如，顺序表删除所有值为 `x` 的元素，可以想到“保留不等于 `x` 的元素”。让指针 `k` 表示新表中下一个可写入位置，扫描指针 `i` 从前到后遍历原表：

```
k = 0;
for (i = 0; i < L.length; i++) {
    if (L.data[i] != x) {
        L.data[k] = L.data[i];
        k++;
    }
}
L.length = k;
```

这个算法的关键不是“删除”，而是“覆盖保留”。所有元素只扫描一次，因此时间复杂度为 $O(n)$ ；只使用常数个变量，因此空间复杂度为 $O(1)$ 。

再例如，单链表逆置常用头插法。核心思想是：从原链表中依次摘下结点，把它插入到头结点之后。

```
p = L->next;
L->next = NULL;
while (p != NULL) {
    r = p->next;
    p->next = L->next;
    L->next = p;
```

```
p = r;
}
```

这里最关键的一句是 `r = p->next`。如果没有先保存后继结点，就直接修改 `p->next`，原链表后续部分就会丢失。链表算法题常考的正是这种“指针修改顺序”。

本章复习重点压缩

本章最终需要掌握的内容可以压缩成下面几组对比：

对比项	必须掌握的结论
线性表与顺序表、链表	线性表是逻辑结构；顺序表和链表是存储结构
顺序表与链表	顺序表支持随机访问；链表插删灵活但不能随机访问
头结点与头指针	头指针指向链表第一个结点；头结点是人为附加的辅助结点
头插法与尾插法	头插法建立逆序链表；尾插法建立正序链表
单链表与双链表	双链表多一个前驱指针，删除和前向遍历更方便
循环链表与普通链表	循环链表尾结点指回头部，适合循环处理
静态链表与动态链表	静态链表用数组下标模拟指针，适合没有指针的环境

从考试角度看，本章最应优先掌握的是：

1. 线性表的定义和基本操作。
2. 顺序表插入、删除、查找的过程与复杂度。
3. 单链表的头插法、尾插法、按序号查找、按值查找、插入和删除。
4. 双链表插入和删除时四条指针语句的先后关系。
5. 循环链表、静态链表的基本思想。
6. 顺序表与链表的适用场景比较。
7. 算法题中常见的双指针、逆置、归并、快慢指针思想。

思维拓展

有序数组中寻找和为 x 的两个数

题目背景可以概括为：给定一个长度为 n 的整型数组 $A[1 \dots n]$ 和整数 x ，设计算法查找数组中是否存在两个元素，它们的和等于 x ，并要求时间复杂度不超过 $O(n \log_2 n)$ 。

这类题的关键是把“任意两个数配对”转化为“有序结构上的双端扫描”。

基本思路如下：

1. 先对数组 $A[1 \dots n]$ 从小到大排序。排序可采用时间复杂度为 $O(n \log_2 n)$ 的算法。
2. 设置两个指针：
 - $i = 1$ ，指向当前最小端；
 - $j = n$ ，指向当前最大端。
3. 比较 $A[i] + A[j]$ 与 x ：

- 若 $A[i] + A[j] < x$ ，说明当前最小数太小，应令 $i++$ 。
- 若 $A[i] + A[j] > x$ ，说明当前最大数太大，应令 $j--$ 。
- 若 $A[i] + A[j] = x$ ，则找到一组解，输出 $A[i]$ 和 $A[j]$ ，然后令 $i++$ 、 $j--$ ，继续寻找其他解。

4. 当 $i \geq j$ 时停止。

伪代码可写为：

```
Sort(A, n);           // 按升序排序

i = 1;
j = n;
while (i < j) {
    sum = A[i] + A[j];
    if (sum < x) {
        i++;
    } else if (sum > x) {
        j--;
    } else {
        printf(A[i], A[j]);
        i++;
        j--;
    }
}
```

这个算法的正确性来自有序性：

- 若当前最小值 $A[i]$ 与最大值 $A[j]$ 相加仍小于 x ，则 $A[i]$ 与任何不超过 $A[j]$ 的数相加都不可能达到 x ，所以可以安全丢弃 $A[i]$ 。
- 若当前和大于 x ，则 $A[j]$ 与任何不小于 $A[i]$ 的数相加都会偏大，所以可以安全丢弃 $A[j]$ 。

复杂度分析：

阶段	时间复杂度	说明
排序	$O(n \log_2 n)$	主导时间开销
双指针扫描	$O(n)$	每次至少移动一个指针
总时间复杂度	$O(n \log_2 n)$	满足题目要求

若排序算法需要额外空间，则空间复杂度取决于所选排序方法；若使用原地排序，额外空间可控制在 $O(1)$ 或递归栈开销范围内。

这道题体现了顺序表算法的一个重要思想：**先利用排序制造有序性，再用双指针降低枚举复杂度**。原始暴力做法要枚举所有二元组，时间复杂度为 $O(n^2)$ ；排序加双指针后，整体降为 $O(n \log_2 n)$ 。这类思想在数组题、顺序表题、双指针题中非常常见。

第三章 栈、队列和数组

本章讨论栈、队列、数组及特殊矩阵压缩存储。栈和队列本质上都是操作受限的线性表：栈只允许在同一端插入和删除，体现后进先出；队列在一端插入、另一端删除，体现先进先出。数组部分的重点是多维数组到一维地址空间的映射，以及特殊矩阵如何利用元素分布规律节省存储空间。

3.1 栈

3.1.1 栈的基本概念

栈是只允许在一端进行插入和删除操作的线性表。允许操作的一端称为栈顶，另一端称为栈底。插入元素称为入栈，删除元素称为出栈。栈的核心特性是后进先出，即 LIFO。

考点追踪：栈的特点（2017）

出入栈序列是本节重点。若元素按固定顺序入栈，出栈序列不是任意排列，必须满足“栈顶元素才能先出”的限制。判断序列是否合法，最稳妥的方法是模拟：按入栈顺序不断压栈，当栈顶等于目标出栈序列当前元素时立即出栈；若栈顶不匹配且无元素可继续入栈，则该序列不合法。

考点追踪：出入栈序列的分析（2010、2011、2013、2018、2020、2022）

当有 n 个不同元素按固定顺序入栈，合法出栈序列个数为 Catalan 数：

$$\frac{1}{n+1} C_{2n}^n$$

该公式只适用于“入栈顺序固定、每个元素入栈和出栈各一次、出栈时机任意”的标准模型。

考点追踪：Catalan 数的应用（2015）

3.1.2 栈的顺序存储结构

顺序栈用连续数组存储栈中元素，并用 `top` 表示栈顶位置。常见约定是：`top` 指向当前栈顶元素，空栈时 `top = -1`。

```
#define MaxSize 50
typedef int ElemType;

typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
```

在该约定下：初始化为 `S.top = -1`；栈空为 `S.top == -1`；栈满为 `S.top == MaxSize - 1`；入栈语句为 `S.data[++S.top] = x`；出栈语句为 `x = S.data[S.top--]`。

顺序栈题要特别注意：数组中残留的旧值不代表当前栈中仍有该元素，当前有效范围只由 `top` 决定。共享栈利用两个栈底分别位于数组两端、两个栈顶向中间增长的特点，让两个栈共享空间。初始时 `top0 = -1`，`top1 = MaxSize`，栈满条件为 `top1 - top0 == 1`。

3.1.3 栈的链式存储结构

链栈通常用单链表实现，并规定链表表头为栈顶。表头插入相当于入栈，表头删除相当于出栈。链栈便于动态分配空间，通常不受固定容量限制，但每个结点需要额外存储指针域。顺序栈和链栈的入栈、出栈时间复杂度都为 $O(1)$ 。

3.2 队列

3.2.1 队列的基本概念

队列只允许在一端插入元素，在另一端删除元素。插入端称为队尾，删除端称为队头。队列的核心特性是先进先出，即 FIFO。栈和队列都是操作受限的线性表，但栈在同一端操作，队列在两端分别操作。

3.2.2 队列的顺序存储结构

顺序队列用连续数组存储元素，并设置 `front` 和 `rear` 两个指针。常见约定是：`front` 指向队头元素，`rear` 指向队尾元素的下一个位置，空队列时 `front == rear`。

普通顺序队列会产生假溢出：数组前部已经空出，但 `rear` 到达数组末尾后不能继续右移。循环队列把数组逻辑上看成环，通过取模实现指针回绕：

```
Q.front = (Q.front + 1) % MaxSize;
Q.rear  = (Q.rear  + 1) % MaxSize;
```

循环队列中元素个数为：

$$\text{len} = (Q.\text{rear} - Q.\text{front} + \text{MaxSize}) \bmod \text{MaxSize}$$

循环队列最容易混淆的是判空和判满。若采用牺牲一个存储单元的方法，判空条件为 `Q.front == Q.rear`，判满条件为 `(Q.rear + 1) % MaxSize == Q.front`。此时容量为 `MaxSize` 的数组最多只能存放 `MaxSize - 1` 个元素。

3.2.3 队列的链式存储结构

链队列本质上是带队头指针和队尾指针的单链表。带头结点的链队列最常见：空队列时 `front == rear`，二者都指向头结点。入队时在队尾插入新结点并移动 `rear`；出队时删除头结点之后的第一

个数据结点。若删除的是最后一个结点，还要让 `rear` 重新指向头结点。

考点追踪：链式队列的应用场景（2019）

考点追踪：链式队列判空的条件（2019）

考点追踪：链式队列出队、入队操作的基本过程（2019）

3.2.4 双端队列

双端队列允许两端都进行插入和删除。输入受限的双端队列限制一端不能插入，输出受限的双端队列限制一端不能删除。此类题常考输出序列是否合法，判断时同样应模拟操作过程。

考点追踪：双端队列输出序列的合法性（2014）

3.3 栈和队列的应用

3.3.1 栈在括号匹配中的应用

括号匹配利用栈保存尚未匹配的左括号。扫描表达式时，遇到左括号就入栈，遇到右括号就检查栈顶是否为对应左括号；若匹配则出栈，否则匹配失败。扫描结束后若栈空，则括号完全匹配。

3.3.2 栈在表达式求值中的应用

表达式求值常涉及中缀、前缀、后缀表达式。后缀表达式求值可用栈完成：遇到操作数就入栈，遇到运算符就从栈中弹出所需操作数并计算，再把结果压回栈。中缀表达式转后缀表达式时，栈用于暂存运算符并处理优先级和括号。

3.3.3 栈在递归中的应用

递归调用由系统栈支持。每一层调用都需要保存返回地址、参数和局部变量。由于后调用的函数先返回，递归天然符合栈的后进先出。递归代码简洁，但递归深度过大可能导致栈溢出，也可能产生大量重复计算。任何递归算法都可以借助显式栈改写为非递归算法。

3.3.4 队列在层次遍历中的应用

层次遍历使用队列保持“先发现先访问”的顺序。以二叉树为例，先让根结点入队；之后循环取出队头并访问，再把其左、右孩子依次入队。由于队列先进先出，结点会按层次顺序被访问。

3.3.5 队列在计算机系统中的应用

队列常用于缓冲区和资源调度。缓冲区解决主机与外设速度不匹配的问题，例如打印缓冲区按写入顺序保存待打印数据。资源调度中，多个请求按到达顺序排队，先到先服务，体现公平性。

考点追踪：缓冲区的逻辑结构（2009）

考点追踪：多队列出队/入队操作的应用（2016）

3.4 数组和特殊矩阵

3.4.1 数组的定义

数组是由 n 个相同类型的数据元素组成的有限序列，下标用于标识元素位置。数组可以看作线性表的推广：一维数组是线性表，二维数组可以看作元素也是一维数组的线性表。数组维数和维界通常固定，主要操作是按下标读取或修改元素。

3.4.2 数组的存储结构

数组通常映射到连续存储空间。设每个元素占 L 个存储单元，一维数组 $A[0..n-1]$ 中元素 $A[i]$ 的地址为：

$$\text{LOC}(a_i) = \text{LOC}(a_0) + iL$$

二维数组可按行优先或列优先展开。若行下标为 $[0, h_1]$ ，列下标为 $[0, h_2]$ ，按行优先时：

$$\text{LOC}(a_{i,j}) = \text{LOC}(a_{0,0}) + [i(h_2 + 1) + j]L$$

按列优先时：

$$\text{LOC}(a_{i,j}) = \text{LOC}(a_{0,0}) + [j(h_1 + 1) + i]L$$

考点追踪：二维数组按行优先存储的下标对应关系（2021）

3.4.3 特殊矩阵的压缩存储

压缩存储的核心是只存必须单独保存的元素。对称矩阵满足 $a_{ij} = a_{ji}$ ，只需存储下三角或上三角加主对角线，共需 $n(n+1)/2$ 个存储单元。若存下三角且矩阵下标从 1 开始、压缩数组 B 下标从 0 开始，则：

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \\ \frac{j(j-1)}{2} + i - 1, & i < j \end{cases}$$

考点追踪：对称矩阵压缩存储的下标对应关系（2018、2020）

三角矩阵除三角部分外，另一半通常全为同一常量，因此还要额外保存该常量一次，总空间为 $n(n+1)/2 + 1$ 。上三角矩阵按行优先存储时，先数前面完整行的上三角元素，再数本行偏移。

考点追踪：上三角矩阵采用行优先存储的应用（2011）

三对角矩阵的非零元素集中在主对角线及其上下相邻两条对角线上。若按行优先存储非零元素，且 a_{11} 存于 $B[0]$ ，则非零元素 a_{ij} 的压缩下标为：

$$k = 2i + j - 3$$

反向映射为：

$$i = \left\lfloor \frac{k+1}{3} \right\rfloor + 1$$

$$j = k - 2i + 3$$

考点追踪：三对角矩阵压缩存储的下标对应关系（2016）

3.4.4 稀疏矩阵

若矩阵中非零元素个数 t 相对于总元素个数 s 很少，即 $t \ll s$ ，称为稀疏矩阵。稀疏矩阵不能只保存非零元素值，还必须保存其行号和列号，通常用三元组 (i, j, a_{ij}) 表示。三元组表可以用数组存储，也可以用十字链表存储；同时还要保存矩阵行数、列数和非零元素个数。

考点追踪：存储稀疏矩阵需要保存的信息（2023）

考点追踪：稀疏矩阵压缩存储的方式（2017）

3.5 本章归纳总结与思维拓展

3.5.1 本章归纳总结

本章的核心不是背代码，而是理解三类结构的限制和用途：栈解决最近状态、回退状态和递归状态；队列解决按到达先后处理的问题；数组解决逻辑下标到物理地址的映射问题。算法设计题中，栈和队列常作为辅助工具，通常可以直接声明结构并写关键操作，不必完整实现所有接口。

```
ElemType stack[maxSize];
int top = -1;
stack[++top] = x;
x = stack[top--];
```

3.5.2 思维拓展：支持 $O(1)$ 最小值查询的栈

要让 `Push`、`Pop` 和 `Min` 都在 $O(1)$ 时间内完成，可以使用两个同步栈：主栈保存元素，辅助栈 `stack_min` 保存每个高度下的当前最小值。入栈时，主栈压入 x ，辅助栈压入 $\min(x, \text{stack_min 栈顶})$ ；出栈时两个栈同步弹出；查询最小值时直接返回辅助栈栈顶。

该方法用 $O(n)$ 的额外空间换取所有操作的 $O(1)$ 时间，体现了栈类题常见的“辅助栈保存历史状态”思想。

当前进度：已阅读并整理《数据结构-第1-5章-绪论至树二叉树.pdf》第 4 章第 49-59 页内容，完成“第四章 串”的全部正文内容，包括 4.1 串的定义和实现、4.2 串的模式匹配、KMP 算法、next 数组、KMP 匹配函数、nextval 优化、本章归纳总结与思维拓展。

下次任务：本章已完成，无需继续追加第 4 章内容；后续可进入下一章或按 workflow 结束当前 act。

第四章 串

本章在考研大纲中的核心要求是**字符串模式匹配**，尤其是 KMP 算法。串的基本概念和存储结构主要用于理解后续算法，不宜死记零散术语；真正需要重点掌握的是：

- 串、子串、主串、位置、串长等基本概念；
- 串与线性表的关系：逻辑结构类似，但基本操作对象通常不同；
- 串的顺序存储、堆分配存储、块链存储的基本思想；
- 简单模式匹配算法的执行过程、最坏时间复杂度；
- 为后续 KMP 做铺垫：简单匹配为什么会产生大量重复比较。

本章内容在大纲中篇幅不大，但 KMP 是典型高频算法考点。复习时应把“模式串如何移动、主串指针是否回溯、next 数组如何产生”作为主线。

4.1 串的定义和实现

字符串简称**串**，是计算机中处理非数值数据的重要对象。信息检索、文本编辑、问答系统、自然语言处理等问题，本质上都离不开对字符串的存储、比较、查找、截取和连接。

从数据结构角度看，串可以理解为一种特殊线性结构：

- 线性表中的每个元素可以是任意数据元素；
- 串中的每个元素被限定为字符；
- 线性表的常见操作常以“单个元素”为单位；
- 串的常见操作通常以“子串”为单位。

因此，串不是因为结构复杂而难，而是因为**操作粒度从单个元素转向一段连续字符**，这直接引出了模式匹配问题。

4.1.1 串的定义

串是由零个或多个字符组成的有限序列，通常记为：

$$S = 'a_1a_2 \cdots a_n' \quad (n \geq 0)$$

其中：

概念	含义	易错点
串名	如式中的 S	串名不是串值本身，而是标识这个串的名字
串值	单引号括起来的字符序列	串值中的字符可以是字母、数字或其他字符
串长	串中字符个数 n	空格也是字符，会计入长度
空串	长度为 0 的串	通常用 $\varnothing$ 表示

概念	含义	易错点
空格串	由一个或多个空格组成的串	不是空串，长度等于空格个数
子串	串中任意多个连续字符组成的序列	必须连续，不连续字符不能称为子串
主串	包含某个子串的串	讨论匹配时，待查找的大串通常是主串
位置	字符在串中的序号	教材中通常以第 1 个字符作为起始位置

例如设：

$$A = 'China\ Beijing', \quad B = 'Beijing', \quad C = 'China'$$

则：

- A 的长度为 13；
- B 的长度为 7；
- C 的长度为 5；
- B 和 C 都是 A 的子串；
- B 在 A 中的位置为 7；
- C 在 A 中的位置为 1。

两个串相等，需要同时满足两个条件：

1. 两个串长度相等；
2. 对应位置上的字符完全相同。

这里要特别区分“空串”和“空格串”：空串没有任何字符，长度为 0；空格串由空格字符组成，只是显示上不明显，本质上仍有字符。

4.1.2 串的基本操作

教材列出的基本操作可以分为“创建/销毁、判断/求值、加工/定位”三类。掌握这些操作的关键不是背函数名，而是理解每个操作对串做了什么。

操作	作用	说明
<code>StrAssign(&T, chars)</code>	赋值	将串 <code>T</code> 赋值为 <code>chars</code>
<code>StrCopy(&T, S)</code>	复制	将串 <code>S</code> 复制给串 <code>T</code>
<code>StrEmpty(S)</code>	判空	若 <code>S</code> 为空串，返回 <code>TRUE</code> ；否则返回 <code>FALSE</code>
<code>StrCompare(S, T)</code>	比较	若 <code>S > T</code> 返回正值；若 <code>S = T</code> 返回 0；若 <code>S < T</code> 返回负值
<code>StrLength(S)</code>	求长	返回串 <code>S</code> 中字符个数
<code>SubString(&Sub, S, pos, len)</code>	求子串	用 <code>Sub</code> 返回串 <code>S</code> 中从第 <code>pos</code> 个字符起、长度为 <code>len</code> 的子串
<code>Concat(&T, S1, S2)</code>	串联接	用 <code>T</code> 返回由 <code>S1</code> 和 <code>S2</code> 拼接而成的新串

操作	作用	说明
<code>Index(S, T)</code>	定位	若主串 <code>S</code> 中存在与串 <code>T</code> 相同的子串，则返回其首次出现的位置；否则返回 0
<code>ClearString(&S)</code>	清空	将串 <code>S</code> 置为空串
<code>DestroyString(&S)</code>	销毁	销毁串 <code>S</code>

其中，教材特别强调：

- `StrAssign`、`StrCompare`、`StrLength`、`Concat`、`SubString` 是串类型的**最小操作子集**；
- 这些操作无法通过其他串操作实现，是构造其他操作的基础；
- 其他串操作通常可基于这个最小操作子集实现；
- `ClearString` 和 `DestroyString` 属于清理类操作，通常不归入构造其他操作的核心最小集。

理解方式：

- `SubString` 负责“截取一段”；
- `Concat` 负责“拼接两段”；
- `Index` 负责“在主串中查找模式串”；
- 后续模式匹配算法本质上就是在更高效地实现 `Index(S, T)`。

4.1.3 串的存储结构

串的存储结构主要有三种：定长顺序存储、堆分配存储、块链存储。考研中更常用它们作为理解背景，重点一般不在复杂实现细节，而在“空间是否连续、长度是否固定、是否便于扩展”。

1. 定长顺序存储表示

定长顺序存储类似顺序表：用一组地址连续的存储单元保存串中的字符，并预先为每个串变量分配固定长度的数组。

```
#define MAXLEN 255           // 预定义最大串长为 255
typedef struct {
    char ch[MAXLEN];         // 每个分量存储一个字符
    int length;               // 串的实际长度
} SString;
```

它的特点是：

- 存储空间连续，便于按下标随机访问字符；
- 串的最大长度由 `MAXLEN` 限定；
- 若实际串长超过 `MAXLEN`，超出部分会被舍弃，称为**截断**；
- 串长可以用 `length` 字段显式保存，也可以在串值末尾添加特殊结束标记。

两种表示串长的方式对比如下：

表示方式	思想	优点	缺点
显式长度 字段	用 <code>length</code> 记录实际长度	求串长方便，适合数据结构教材中的抽象操作	多维护一个字段
结束标记	在串尾添加不计入串长的特殊字符	接近 C 语言字符串思想	求长度通常需要从 头扫描

当执行插入、联接等操作时，如果结果串长度超过 `MAXLEN`，就必须截断。为了从根本上避免这个硬性上限，可采用动态分配方式。

2. 堆分配存储表示

堆分配存储仍然用地址连续的存储单元保存字符序列，但空间不是编译时固定分配，而是在程序运行过程中动态申请。

```
typedef struct {
    char *ch;           // 按串长分配存储区，ch 指向串的基地址
    int length;         // 串的长度
} HString;
```

在 C 语言中，程序有一个称为“堆”的自由存储区，可以通过 `malloc()` 动态申请空间，通过 `free()` 释放空间。

堆分配存储的基本过程是：

1. 根据串长申请一块连续空间；
2. 将 `ch` 指向这块空间的起始地址；
3. 若申请成功，就把字符依次存入；
4. 若申请失败，返回 `NULL`；
5. 串不再使用时，通过 `free()` 释放空间。

与定长顺序存储相比，堆分配存储的优势是更灵活，能根据实际串长动态改变所需空间。因此，很多高级程序设计语言会采用类似思想实现字符串。

3. 块链存储表示

块链存储类似线性表的链式存储，但串的每个结点不一定只存一个字符，而是可以存多个字符。若每个结点存多个字符，就称为**块链结构**。

教材中的示意可以用下面的形式理解：

```
结点大小为 4：
head -> [A B C D] -> [E F G H] -> [I # # # ^]

结点大小为 1：
head -> [A] -> [B] -> [C] -> ... -> [I ^]
```

其中：

- 每个结点能存放的字符数称为**结点大小**；

- 当最后一个结点没有存满时，可用特殊字符如 # 补齐；
- ^ 可以理解为串结束标志；
- 若结点大小为 1，则每个结点只存一个字符，结构更接近普通链表；
- 若结点大小较大，则能减少指针域带来的空间浪费。

三种存储结构可以这样比较：

存储方式	空间连续性	长度限制	主要优点	主要缺点
定长顺序存储	连续	固定上限	简单、随机访问方便	可能截断，空间不够灵活
堆分配存储	连续	动态申请	灵活，能按需分配	需要管理动态内存
块链存储	分块连续，块间链式	较灵活	便于扩展，适合很长的串	随机访问不如顺序存储方便，指针有额外开销

复习时不必把块链存储的图机械记下来，只要把握一句话：**串也可以链式存储，但为了减少指针开销，常把多个字符放在同一个结点中。**

4.2 串的模式匹配

模式匹配是本章的核心。所谓模式匹配，是指在主串中查找与模式串完全相同的子串，并返回其首次出现的位置。

设：

- 主串为 S ；
- 模式串为 T ；
- 若 T 在 S 中出现，则返回首次出现的起始位置；
- 若没有出现，则返回 0。

从抽象操作角度看，模式匹配就是 $\text{Index}(S, T)$ 的具体实现。

4.2.1 简单的模式匹配算法

简单模式匹配算法也称暴力匹配算法。它的基本思想非常直接：

1. 从主串 S 的第一个字符开始，与模式串 T 的首字符比较；
2. 若当前字符相等，则主串指针和模式串指针都后移，继续比较后续字符；
3. 若某个字符不相等，则本次匹配失败；
4. 主串的比较起点后移一位，模式串指针重新回到首字符；
5. 重复上述过程，直到匹配成功或主串剩余长度不足。

教材给出的算法基于定长顺序存储结构，可写成：

```
int Index(SString S, SString T) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) {
            ++i;
            ++j;           // 继续比较后继字符
        } else {
            i = i - j + 2;
            j = 1;         // 指针后退，重新开始匹配
        }
    }
    if (j > T.length)
        return i - T.length; // 匹配成功，返回起始位置
    else
        return 0;           // 匹配失败
}
```

这里最容易混淆的是失配后的语句：

```
i = i - j + 2;
j = 1;
```

其含义是：

- 当前已经比较了 $j - 1$ 个字符；
- 本趟匹配的起始位置是 $i - j + 1$ ；
- 下一趟匹配要从主串起始位置的后一位开始；
- 因此新的主串指针为 $(i - j + 1) + 1 = i - j + 2$ ；
- 模式串重新从第一个字符开始比较，所以 $j = 1$ 。

教材中的例子为：

$$T = 'abcac'$$

主串可理解为：

$$S = 'ababcabcacbab'$$

简单匹配过程大致如下：

趟数	主串当前起点	比较结果	下一步
第 1 趟	1	前两个字符匹配，到第 3 个字符失配	起点后移到 2，模式串回到首字符
第 2 趟	2	首字符即失配	起点后移到 3
第 3 趟	3	匹配到中途失配	起点后移到 4
第 4 趟	4	首字符失配	起点后移到 5
第 5 趟	5	首字符失配	起点后移到 6
第 6 趟	7	模式串所有字符依次匹配	匹配成功，返回起始位置 7

这个例子要看出的重点不是每个箭头的位置，而是：**每次失配后，简单匹配都会把模式串整体只右移一位，并且主串指针需要回溯到新的起点。**

设主串长度为 n ，模式串长度为 m ，且 $n \gg m$ 。简单模式匹配算法最多需要进行：

$$n - m + 1$$

趟匹配，每趟最多比较 m 次，所以最坏时间复杂度为：

$$O(nm)$$

教材举出的最坏情况可以概括为：

- 模式串形如 0000001；
- 主串前面有大量连续的 0，末尾才出现 1 或根本不能匹配；
- 每一趟都要比较到模式串最后一个字符附近才发现失配；
- 大量已经比较过的信息被丢弃，导致重复比较。

例如模式串长度为 7，若主串中连续许多字符都是 0，简单匹配可能每一趟都比较很多次，主串指针还会不断回溯。教材例子中主串指针需要回溯 39 次，总比较次数达到 $40 \times 7 = 280$ 次。

这正是 KMP 算法要解决的问题：

简单模式匹配的低效不在于“比较字符”本身，而在于失配后没有利用已经匹配成功的前缀信息，导致主串指针反复回溯。

4.2.2 串的模式匹配算法——KMP 算法

KMP 算法要解决的是简单模式匹配中的重复比较问题。简单模式匹配每次失配后，通常会让主串指针回退到下一趟匹配起点，模式串也回到首字符，这会浪费已经比较成功的信息。

以教材前面的例子为线索：主串为 `ababcabcacbab`，模式串为 `abcac`。在第三趟匹配中，若比较到 `i = 7, j = 5` 时发生失配，简单模式匹配会回退到 `i = 4, j = 1` 重新开始。可是从已匹配结果已经知道：主串中第 4、5、6 个字符 `bca` 与模式串中第 2、3、4 个字符 `bca` 相同，而模式串首字符是 `a`，所以让模式串首字符再去和主串的 `b`、`c`、`a` 逐个比较，就会产生多余操作。

KMP 的核心思想是：

- 主串指针不回溯；
- 失配后只根据模式串自身结构调整模式串指针；
- 已经匹配成功的前缀信息不丢弃，而是转化为模式串下一次比较的位置。

也就是说，KMP 的难点不在“字符是否相等”，而在“失配后模式串应该跳到哪里”。

1. KMP 算法的原理：前缀、后缀与部分匹配值

理解 KMP 前，先要理解三个概念。

概念	含义	注意点
前缀	除最后一个字符外，字符串的所有头部子串	不能包含整个字符串本身
后缀	除第一个字符外，字符串的所有尾部子串	不能包含整个字符串本身
部分匹配值	前缀和后缀的最长相等前后缀长度	本质是“首尾能重合多长”

以字符串 `ababa` 为例：

子串	前缀集合	后缀集合	最长相等前后缀	部分匹配值
<code>a</code>	空集	空集	无	0
<code>ab</code>	<code>{a}</code>	<code>{b}</code>	无	0
<code>aba</code>	<code>{a, ab}</code>	<code>{a, ba}</code>	<code>a</code>	1
<code>abab</code>	<code>{a, ab, aba}</code>	<code>{b, ab, bab}</code>	<code>ab</code>	2
<code>ababa</code>	<code>{a, ab, aba, abab}</code>	<code>{a, ba, aba, baba}</code>	<code>aba</code>	3

所以，模式串 `ababa` 的部分匹配值序列为：

0 0 1 2 3

对于教材中的模式串 `abcac`，各位置的部分匹配值为：

编号	1	2	3	4	5
模式串	a	b	c	a	c

编号	1	2	3	4	5
PM	0	0	0	1	0

PM 表的作用是决定失配后模式串应该右移多少位：

$$\text{右滑位数} = \text{已匹配字符数} - \text{对应的部分匹配值}$$

例如模式串 `abcac` 在第一趟匹配时，前两个字符 `ab` 已匹配，第三个字符发生失配。最后一个已匹配字符是 `b`，它对应的部分匹配值为 0，因此：

$$\text{右滑位数} = 2 - 0 = 2$$

第二趟匹配时，若已经匹配 `abca` 后在第 5 个字符失配，最后一个已匹配字符是 `a`，对应的部分匹配值为 1，因此：

$$\text{右滑位数} = 4 - 1 = 3$$

这种移动方式的含义是：若已匹配部分的后缀恰好等于模式串的前缀，就把模式串移动到两者对齐的位置，直接跳过已经确定相等的那一段，避免重复比较。

因此，KMP 的稳定时间复杂度可以达到：

$$O(n + m)$$

其中， n 是主串长度， m 是模式串长度。和简单模式匹配的 $O(nm)$ 相比，KMP 的优势在于**失配后不回溯主串指针**。

2. next 数组的手算方法

在实际匹配过程中，模式串在内存中的位置并不会真的整体移动，发生变化的是指针。为了描述模式串指针失配后跳到哪里，教材引入 `next` 数组。

`next[j]` 的含义是：

当模式串的第 j 个字符与主串当前字符失配时，下一轮应让模式串的第 `next[j]` 个字符继续与主串当前字符比较。

这里仍以模式串 `abcac` 为例，按 1 开始编号。

- 第 1 个字符失配时，规定 `next[1] = 0`。这表示模式串首字符都不能匹配，需要让主串指针后移一位，并让模式串重新从第 1 个字符开始比较。
- 第 2 个字符失配时，`next[2] = 1`。由于第 1 个字符已经匹配，但不存在可复用的更短前后缀，下一轮回到模式串第 1 个字符比较。
- 第 3 个字符失配时，`next[3] = 1`。
- 第 4 个字符失配时，`next[4] = 1`。
- 第 5 个字符失配时，`next[5] = 2`。因为失配前已匹配的部分中存在可复用的相同前后缀，下一轮可以从模式串第 2 个字符继续比较。

于是得到：

编号	1	2	3	4	5
模式串	a	b	c	a	c
next[j]	0	1	1	1	2

next 数组可以理解为“指针跳转表”。它不是直接告诉模式串右移几位，而是告诉模式串指针下一次落到哪个位置。

考点追踪：KMP 算法中指针变化和比较次数的分析，曾在 2015 年、2019 年考查。复习时不要只会背 next 表，更要能说明失配时主串指针为什么不回溯、模式串指针为什么跳到 next[j]。

3. next 数组与 PM 表的关系

教材给出 next 数组和 PM 表之间的关系。若第 j 个字符失配，则此时已经成功匹配的字符数为 $j - 1$ 。根据 PM 表，模式串右滑位数为：

$$(j - 1) - PM[j - 1]$$

而模式串第 j 个字符失配后，下一次比较的位置为：

$$\begin{aligned} next[j] &= j - \text{右滑位数} \\ &= j - ((j - 1) - PM[j - 1]) \\ &= PM[j - 1] + 1 \end{aligned}$$

因此，可以把 PM 表转化为 next 数组：

1. 将 PM 表整体右移一位；
2. 第一个位置补 0；
3. 右移后每个有效位置整体加 1；
4. 原 PM 表最后一个值被移出，因为最后一个字符的部分匹配值是供“下一个字符”使用的，而最后一个字符之后已经没有下一个字符。

以 abcac 为例：

```
PM:      0 0 0 1 0
右移补 0: 0 0 0 0 1
整体加 1: 0 1 1 1 2
next:    0 1 1 1 2
```

需要特别注意：教材这里采用的是串的编号从 1 开始的写法。若某些资料或代码采用从 0 开始编号，则 next 数组的表示通常会整体减 1，因此看到不同版本时不要机械比较数值，要先确认编号规则。

4. next 数组的推理公式

设主串为 $s_1s_2\cdots s_n$ ，模式串为 $p_1p_2\cdots p_m$ 。当主串第 i 个字符与模式串第 j 个字符失配时，已经匹配成功的是模式串的前 $j - 1$ 个字符。

如果希望下一轮直接让模式串第 k 个字符与主串第 i 个字符比较，那么模式串前 $k - 1$ 个字符必须与失配点前面已经匹配部分的后 $k - 1$ 个字符相等，即：

$$p_1p_2\cdots p_{k-1} = p_{j-k+1}p_{j-k+2}\cdots p_{j-1}$$

在满足这个条件的所有 k 中，应选择最大的 k ，因为这样能保留最长的已匹配信息，也就能让模式串移动得最少但不漏解。

因此，`next[j]` 的一般定义可写为：

$$next[j] = \begin{cases} 0, & j = 1, \\ \max\{k \mid 1 < k < j, p_1\cdots p_{k-1} = p_{j-k+1}\cdots p_{j-1}\}, & \text{该集合非空,} \\ 1, & \text{其他情况.} \end{cases}$$

这个公式看起来抽象，但本质仍是：**找失配位置之前那段字符串的最长相等前后缀，然后让模式串前缀对齐到对应后缀上。**

5. next 数组的递推求解思想

直接按公式求 `next` 数组较麻烦，教材进一步推导出递推求法。设已经知道 `next[j] = k`，也就是说在求第 j 个位置时，已经找到了长度为 $k - 1$ 的相等前后缀。接下来求 `next[j+1]` 时分两种情况。

第一种情况：若 $p_k = p_j$ ，则原来的相等前后缀可以同时向后扩展一个字符，于是：

$$next[j + 1] = k + 1 = next[j] + 1$$

第二种情况：若 $p_k \neq p_j$ ，说明当前长度的相等前后缀无法继续扩展，需要把求 `next` 的问题转化为一个更短的模式匹配问题。此时令 $k = next[k]$ ，继续尝试更短的相等前后缀，直到找到可以扩展的位置；若不存在更短的相等前后缀，则令 `next[j+1] = 1`。

教材给出的例子是模式串 `abaabcaba`。前 6 个位置的 `next` 值为：

编号	1	2	3	4	5	6	7	8	9
模式串	a	b	a	a	b	c	a	b	a
<code>next[j]</code>	0	1	1	2	2	3	?	?	?

继续求后面三个值：

- 求 `next[7]`：已知 `next[6] = 3`，但 $p_6 = c$ ， $p_3 = a$ ，不相等；再回退到 `next[3] = 1`，比较 p_6 与 p_1 ，仍不相等；再回退到 `next[1] = 0`，所以 `next[7] = 1`。
- 求 `next[8]`：因为 $p_7 = p_1$ ，所以 `next[8] = next[7] + 1 = 2`。
- 求 `next[9]`：因为 $p_8 = p_2$ ，所以 `next[9] = next[8] + 1 = 3`。

最终得到：

```
模式串:  a b a a b c a b a
next:    0 1 1 2 2 3 1 2 3
```

6. 求 next 数组的代码实现

根据上述递推思想，可以写出教材中的 `get_next` 函数。这里的实现默认串从 1 开始编号。

```
void get_next(SString T, int next[]) {
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i;
            ++j;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}
```

这段代码可以从两个指针理解：

- `i` 表示当前正在求 `next` 的位置附近；
- `j` 表示当前可复用的最长相等前后缀长度加 1，也就是下一次尝试对齐的位置；
- 若 `j == 0`，说明已经没有可继续回退的前缀，只能让 `i` 后移，令新位置的 `next` 值从 1 开始；
- 若 `T.ch[i] == T.ch[j]`，说明相等前后缀可以扩展，于是 `i`、`j` 同时后移，并令 `next[i] = j`；
- 若 `T.ch[i] != T.ch[j]`，说明当前长度不能扩展，就让 `j = next[j]`，继续尝试更短的相等前后缀。

手算时没有必要完全模拟代码，但要理解代码背后的递推逻辑：**匹配就扩展，失配就沿着 `next` 链回退，直到能扩展或退到 0。**

7. KMP 匹配函数的代码实现

有了 `next` 数组以后，KMP 的匹配过程就可以直接写成代码。与简单模式匹配相比，关键差异只有一点：**主串指针 `i` 不回溯，模式串指针 `j` 按 `next[j]` 回退。**

```
int Index_KMP(SString S, SString T, int next[]) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) {
            ++i;
            ++j;           // 继续比较后继字符
        } else {
            j = next[j];
        }
    }
}
```

```

        j = next[j];          // 模式串向右滑动
    }
}
if (j > T.length)
    return i - T.length;    // 匹配成功
else
    return 0;
}

```

这段代码可以这样理解：

- 当 `S.ch[i] == T.ch[j]` 时，说明当前字符匹配成功，两个指针同时后移；
- 当 `j == 0` 时，说明模式串已经无法再利用任何前缀信息，只能让主串继续前进，并把模式串重新从第一个字符开始尝试；
- 当 `S.ch[i] != T.ch[j]` 时，主串的 `i` 不动，模式串的 `j` 回退到 `next[j]`；
- 一旦 `j > T.length`，说明模式串已经全部匹配成功，此时匹配起始位置为 `i - T.length`；
- 若主串扫描结束仍未匹配完模式串，则返回 `0`。

这里最容易混淆的是返回值。因为循环中每次匹配成功都会先执行 `++i` 和 `++j`，当整个模式串匹配完时，`i` 已经指向匹配子串之后的下一个位置，所以起始位置是：

$$i - T.length$$

8. KMP 与简单模式匹配的复杂度比较

设主串长度为 n ，模式串长度为 m 。

算法	主串指针是否回溯	最坏时间复杂度	说明
简单模式匹配	回溯	$O(mn)$	在大量部分匹配时会反复比较同一批主串字符
KMP 算法	不回溯	$O(m + n)$	通过 <code>next</code> 数组利用已匹配信息，避免无效比较

不过教材也提醒：简单模式匹配虽然最坏复杂度为 $O(mn)$ ，但在一般情况下，平均执行时间常常接近 $O(m + n)$ ，因此实际工程中不一定总能明显感到 KMP 的优势。KMP 真正明显优于暴力匹配的场景，是主串和模式串之间存在大量“部分匹配”时。

复习时应把 KMP 的核心优势压缩成一句话：**KMP 不是让比较本身变快，而是让主串指针不回溯，尽量少做重复比较。**

4.2.3 KMP 算法的进一步优化

考点追踪：nextval 数组的计算（2024）

前面定义的 next 数组已经能保证主串指针不回溯，但在某些特殊模式串中，仍然可能出现“明知还会失配，却仍然继续比较”的情况。因此教材引入 nextval 数组，对 next 数组继续优化。

1. 为什么 next 数组还可能有缺陷

教材以模式串 aaaab 与主串 aaabaaaab 的匹配为例。开始比较时，前 3 个字符都能匹配，到了第 4 个位置出现失配：

```
主串：  a a a b a a a a b
模式串：a a a a b
        1 2 3 4 5
```

此时 $i = 4, j = 4$ ，主串字符为 b，模式串字符为 a，发生失配。若使用普通 next 数组：

```
模式串：      a a a a b
j:           1 2 3 4 5
next[j]:      0 1 2 3 4
nextval[j]:   0 0 0 0 4
```

根据普通 next：

- $next[4] = 3$ ，下一步比较主串 b 与模式串第 3 个字符 a，仍失配；
- $next[3] = 2$ ，继续比较主串 b 与模式串第 2 个字符 a，仍失配；
- $next[2] = 1$ ，继续比较主串 b 与模式串第 1 个字符 a，仍失配；
- 最后回退到 $next[1] = 0$ 。

这些比较其实没有必要，因为模式串第 4、3、2、1 个字符都相同，都是 a。既然已经知道主串当前字符 b 与模式串第 4 个字符 a 不相等，那么它也必然与前面那些同样为 a 的字符不相等。

问题根源可以概括为：当 $p_j = p_{next[j]}$ 时，若 p_j 已经和主串当前字符失配，那么继续让 $p_{next[j]}$ 去比较，只会重复同样的失配。

2. nextval 的优化思想

普通 next 数组只回答一个问题：失配后模式串应该回到哪里？

nextval 数组进一步回答：如果回退位置上的字符和当前失配字符相同，那这个回退是否也没有意义？

优化规则可以这样理解：

- 若 $p_j \neq p_{next[j]}$ ，说明回退后比较的是一个不同字符，有必要尝试，因此 $nextval[j] = next[j]$ ；

- 若 $p_j = p_{next[j]}$ ，说明回退后比较的字符与刚刚失配的字符相同，必然继续失配，因此应继续沿着 `next` 链向前跳；
- 这个过程重复进行，直到找到某个位置 k ，使 $p_j \neq p_k$ ，或者退到 $k = 0$ 为止。

因此，`nextval` 并没有改变 KMP 的整体匹配思想，只是把“会产生无效比较的回退位置”提前跳过。

3. 求 `nextval` 数组的代码实现

教材给出的实现如下：

```
void get_nextval(SString T, int nextval[]) {
    int i = 1, j = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i;
            ++j;
            if (T.ch[i] != T.ch[j])
                nextval[i] = j;
            else
                nextval[i] = nextval[j];
        } else {
            j = nextval[j];
        }
    }
}
```

这段代码与 `get_next` 很像，但多了一个判断：

```
if (T.ch[i] != T.ch[j])
    nextval[i] = j;
else
    nextval[i] = nextval[j];
```

这正是优化的核心：

- 若当前位置字符和回退位置字符不同，就保留当前回退位置；
- 若二者相同，就继续使用更早的 `nextval[j]`，避免重复失配。

对于模式串 `aaaab`：

模式串：	a a a a b
j：	1 2 3 4 5
next[j]：	0 1 2 3 4
nextval[j]：	0 0 0 0 4

前四个位置都是 `a`，所以失配时没必要在这些 `a` 之间反复跳转；第 5 个字符是 `b`，与前面的 `a` 不同，因此它的优化值仍保留为 4。

4. 复习时如何看待 `nextval`

`nextval` 是 KMP 的进一步优化，但不要把它理解成另一个全新的算法。它只是在 `next` 的基础上进一步减少无效比较。

复习优先级建议如下：

1. 先彻底理解简单模式匹配为什么低效；
2. 再理解 KMP 为什么主串指针不回溯；
3. 重点掌握 `next` 数组的含义、手算和代码；
4. 最后再理解 `nextval` 是如何跳过“相同字符导致的必然失配”。

教材也提示：KMP 对初学者来说不容易掌握，建议结合配套课程学习。考研复习时，不要把 KMP 学成纯背代码，而要围绕“失配后模式串跳到哪里”这一核心问题展开。

本章归纳总结

KMP 复习主线

本章真正的核心是字符串模式匹配，尤其是 KMP。学习时可以从暴力匹配的低效性入手：简单模式匹配在失配后会主串指针回溯，从而导致大量重复比较。

KMP 的改进思路是：已匹配的那一段往往恰好包含模式串自身的某个前缀。若失配前的已匹配部分存在“最长相等前后缀”，就可以把模式串滑动到这个前后缀对齐的位置，而不必让主串指针回退。因此，KMP 的本质不是“神奇地跳过字符”，而是利用模式串自身结构，预先计算每个失配位置的最佳跳转位置。`next` 数组记录的是失配时模式串应该回退到的位置；`nextval` 则进一步跳过那些字符相同、必然继续失配的位置。

本章易错点集中整理

易错点	正确理解
空串与空格串混淆	空串长度为 0，空格串由空格字符组成，长度不为 0
串和线性表完全等同	串是一种特殊线性结构，但基本操作通常以子串为单位
简单匹配一定很慢	简单匹配最坏为 $O(mn)$ ，但平均情况常接近 $O(m + n)$
KMP 会让主串和模式串都不回退	KMP 的关键是主串指针不回溯，模式串指针会根据 <code>next</code> 回退
<code>next[j]</code> 只是机械数组	<code>next[j]</code> 本质上来自失配位置之前子串的最长相等前后缀
<code>nextval</code> 是另一个算法	<code>nextval</code> 只是对 <code>next</code> 的优化，用于跳过必然失配的位置

思维拓展

编程实现中，常见扩展问题是：**模式串在主串中有多少个完全匹配的子串？**

这个问题与普通的“返回第一次匹配位置”不同。普通匹配在找到一次后即可结束；若要统计出现次数，则需要在一次匹配成功后继续从后面查找，并处理是否允许重叠匹配的问题。

例如主串为 `aaaaa`，模式串为 `aaa`：

- 若允许重叠匹配，则出现位置为 1、2、3，共 3 次；
- 若不允许重叠匹配，则只统计位置 1，共 1 次。

教材提示统考不考 KMP 算法编程题，但这个问题能帮助理解“匹配成功之后模式串如何继续移动”。掌握思想即可，不需要把它作为本章主考点。